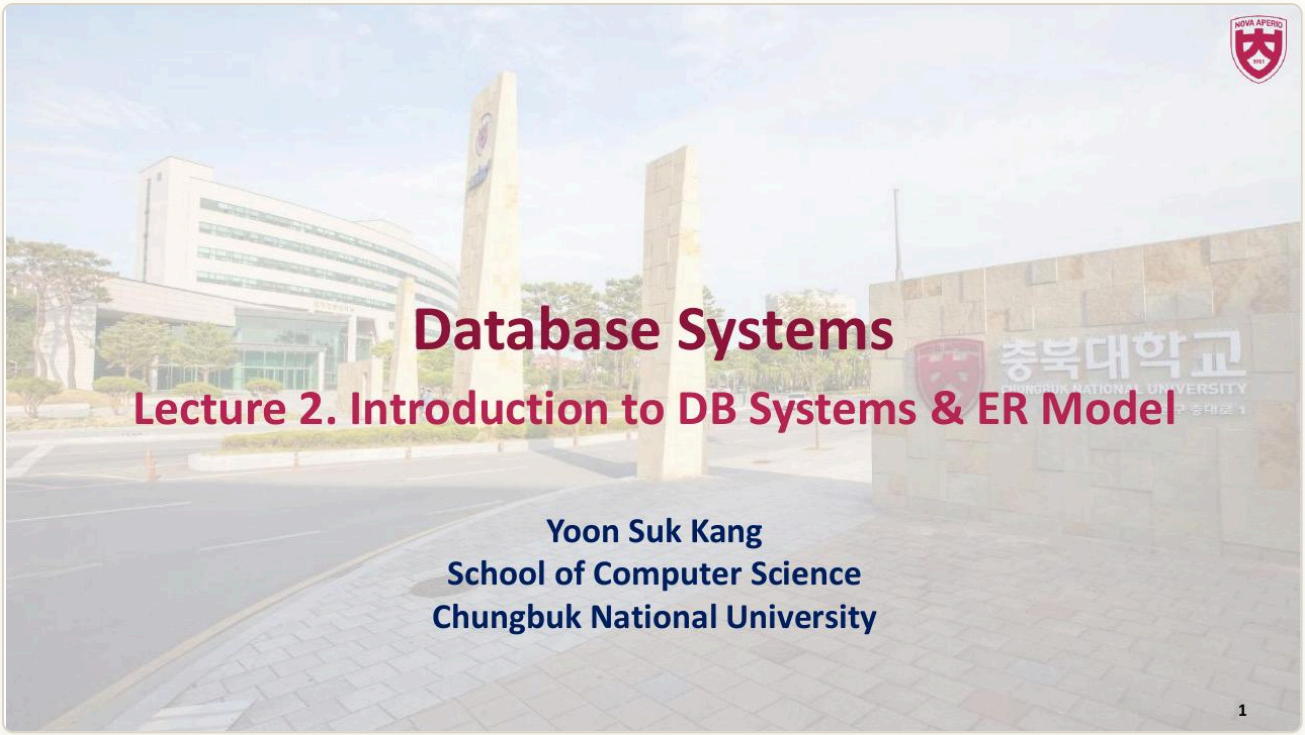
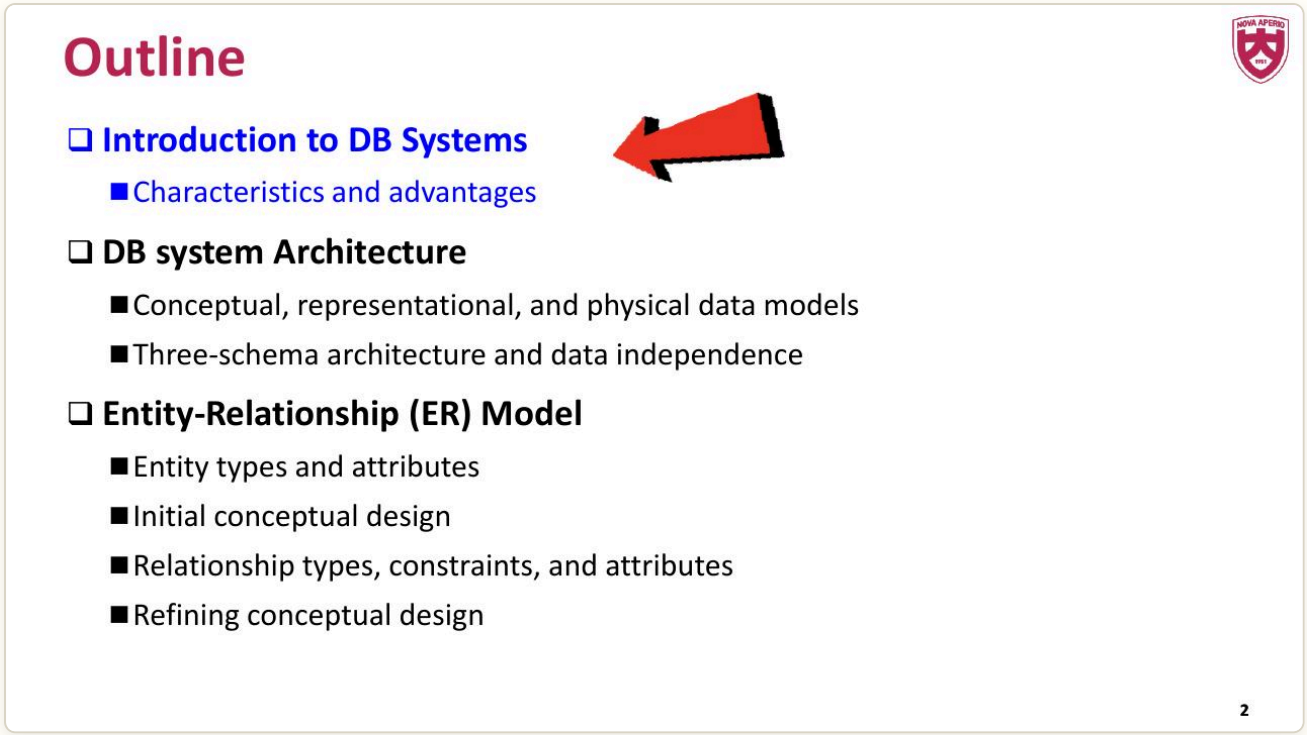


데이터베이스 시스템 — 2강: DB 시스템 소개와 ER 모델

- Yoon Suk Kang · 충북대 소프트웨어학부

필기 ✍️

💡 이해 이 강의 세 덩어리: ① DB 시스템이 뭔가 ② 3단계 스키마 ③ ER 모델로 설계하기



목차 (Outline)

- **DB 시스템 소개** — 특성과 장점
- **DB 시스템 구조** — 개념·표현·물리 데이터 모델 / 3단계 스키마와 데이터 독립성
- **ER 모델** — 엔티티 타입과 속성 / 초기 개념 설계 / 관계 타입·제약·속성 / 개념 설계 다듬기

필기 ✍️

★ 시험 목차 순서 = 시험 범위 순서. ER 모델 비중이 가장 크다 (p.39~81)

What is a Database System?

❑ **Database (DB)**

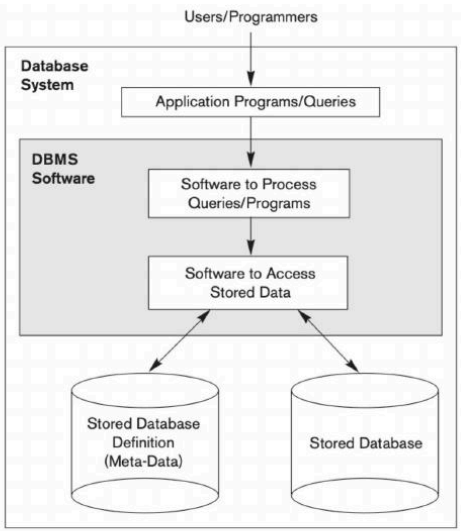
- A logically coherent collection of inter-related data
  - ❑ Managed by a ***Database Management System***

❑ **Database Management System (DBMS)**

- A software that is designed to efficiently "**manage**" data in a structured way
  - ❑ ***Defining***, ***constructing***, ***manipulating***, and ***sharing*** databases
  - ❑ E.g., IBM DB2, Oracle, MS SQL server, MYSQL, SQLite, …

❑ **Database System**

- Database + DBMS software



3

데이터베이스 시스템이란?

- **Database (DB)**: 논리적으로 일관된, 서로 관련된 데이터의 모음
  - DBMS가 관리한다
- **DBMS**: 데이터를 구조적으로 효율 있게 '관리'하도록 만든 소프트웨어
  - DB를 **정의·구축·조작·공유** (defining, constructing, manipulating, sharing)
  - 예: IBM DB2, Oracle, MS SQL Server, MySQL, SQLite
- **Database System** = Database + DBMS 소프트웨어

필기 ✍️

★ 시험 공식처럼: Database System = Database + DBMS

★ 시험 DBMS 하는 일 4개 영어로: Defining · constructing · Manipulating · Sharing

⚠ 주의 DB(데이터 자체) ≠ DBMS(관리 소프트웨어). 섞어 쓰면 틀림

Why Use Database Systems?



❑ **Simple Real-World Example: Message App**

- ■ We want to manage data for a simple message application
- ■ Necessary data
  - ❑ Information of friends: Name, Phone number, isAdvertisement, …
  - ❑ Information of messages: Message texts, sender information, date, …

❑ **Using Simple Text Files**

- ■ Store each line in the file
- ■ Separate each attribute by a comma

Friends (name, number, ad)

"Jonghyeok" , 010-1234-1234 , No
"McDonald" , NULL , Yes
"Starbucks" , NULL , Yes

Message (content, sender, date)

"Class.." , "Jonghyeok", 2409020130
"Today …" , "McDonald" , 2409020900
"Get promo.." , "Starbucks" , 2409020900

4

왜 DB 시스템을 쓸까?

- 간단한 실제 예: **메신저 앱** 데이터를 관리하고 싶다
  - 필요한 데이터 — 친구 정보(이름, 전화번호, 광고 여부…), 메시지 정보(내용, 보낸 사람, 날짜…)
- **그냥 텍스트 파일**로 저장한다면
  - 한 줄에 하나씩, 속성은 쉼표로 구분
  - Friends(name, number, ad) / Message(content, sender, date)

필기 ✍️

💡 이해 메모장에 "Jonghyeok", 010-…, No 처럼 쉼표로 적는 방식 → 다음 쪽에서 문제점이 쏟아진다

Why Use Database Systems? (cont.)

❑ **Data Integrity**
- How can we manage the total length of message?
- What if a user adds duplicate friend information by mistake?
- What if there are friends with the same name?



❑ **Data Management**
- How can we find a particular message quickly?
- What if multiple users try to read/write at the same time?



❑ **Data Durability**
- What if the machine crashes while the app is writing new entries?
- How to restore files after crash?



There are a number of technical issues to be addressed😭
Let's use DBMS!

5

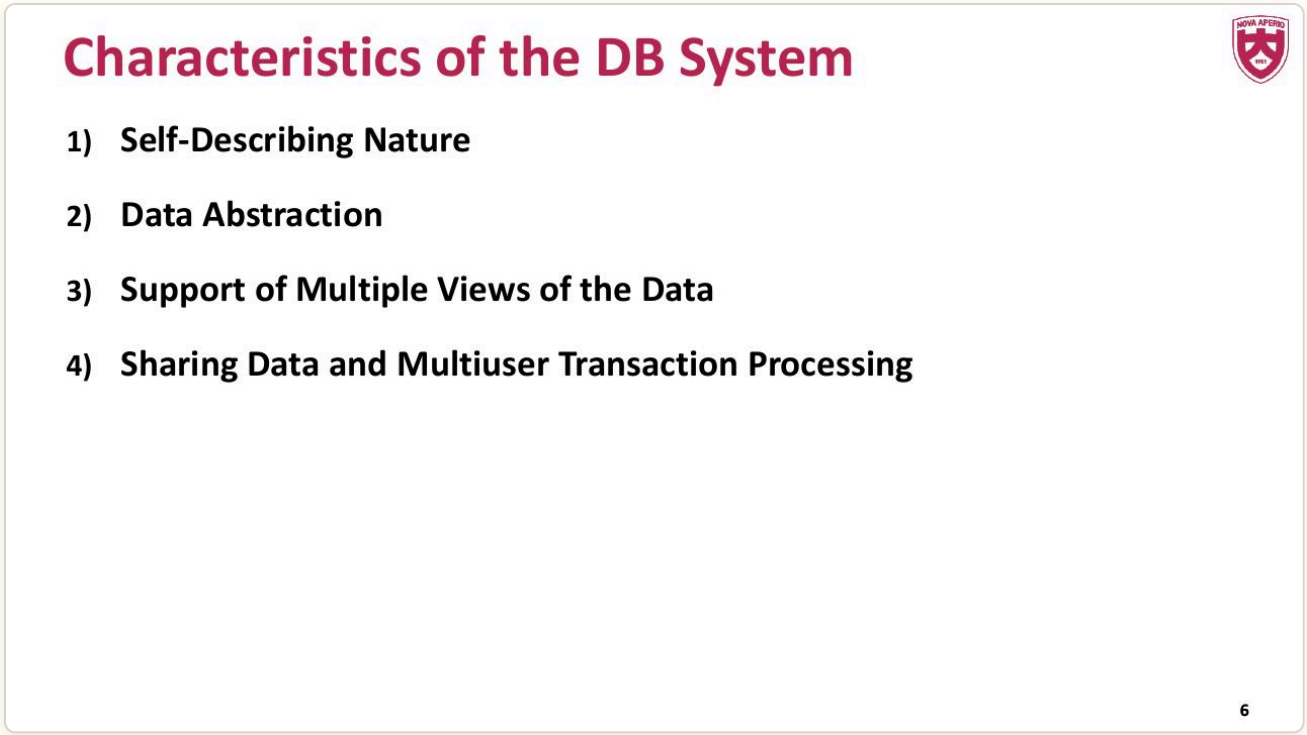
왜 DB 시스템을 쓸까? (계속)

- **Data Integrity (무결성)**
  - 메시지 전체 길이는 어떻게 관리?
  - 친구 정보를 실수로 두 번 넣으면?
  - 이름이 같은 친구가 있으면?
- **Data Management (관리)**
  - 특정 메시지를 빨리 찾으려면?
  - 여러 사용자가 동시에 읽고 쓰면?
- **Data Durability (내구성)**
  - 새 내용을 쓰는 중에 기계가 꺼지면?
  - 고장 뒤 파일을 어떻게 되살리지?
- → 풀어야 할 기술 문제가 많다 😭 DBMS를 쓰자!

필기 ✍️

★ 시험 3범주 영어 그대로: Integrity / Management / Durability

✏ 예.풀이 질문 → 범주 짝: 중복 친구·동명이인 = Integrity, 빠른 검색·동시 접근 = Management, 크래시·복구 = Durability



DB 시스템의 특성

- 1) **Self-Describing Nature** — 자기 기술성
- 2) **Data Abstraction** — 데이터 추상화
- 3) **Support of Multiple Views** — 다중 뷰 지원
- 4) **Sharing Data & Multiuser Transaction Processing** — 데이터 공유와 다중 사용자 트랜잭션 처리

필기 ✍️

★ 시험 특성 4개를 영어 이름으로 외우기 (서술형 단골)

⚠ 주의 '특성(characteristics)' 4개와 p.11 '장점(Advantages)' 4개를 섞지 말 것

1) Self-Describing Nature

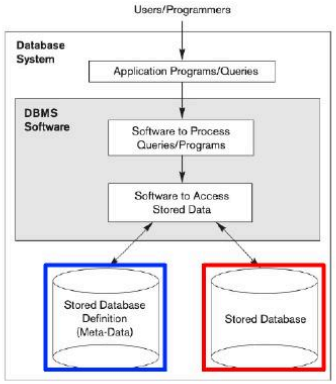
❑ **A Database System Contains the DB Itself and Meta-Data**

■ Meta-data: a definition of the database structure and constraints

❑ Meta-data is stored in the DBMS catalog

❑ It is necessary to maintain the DB

Q. How can we define them? → SQL in Week 4



RELATIONS

Relation_name	No_of_columns
STUDENT	4
COURSE	4
SECTION	5
GRADE_REPORT	3
PREREQUISITE	2

An example of a database catalog

COLUMNS

Column_name	Data_type	Belongs_to_relation
Name	Character (30)	STUDENT
Student_number	Character (4)	STUDENT
Class	Integer (1)	STUDENT
Major	Major_type	STUDENT
Course_name	Character (10)	COURSE
Course_number	XXXXNNNN	COURSE
....	....	.....
....	....	.....
....	....	.....
Prerequisite_number	XXXXNNNN	PREREQUISITE

7

1) 자기 기술성 (Self-Describing Nature)

- DB 시스템은 **DB 자체**와 **메타데이터**를 함께 담는다
  - **Meta-data**: DB의 구조와 제약을 정의한 것
  - 메타데이터는 **DBMS catalog**에 저장된다
  - DB를 유지하는 데 꼭 필요
- 그림: Stored Database Definition(메타데이터) + Stored Database
- 예: 카탈로그의 RELATIONS(표 이름·열 수), COLUMNS(열 이름·자료형·소속 표)
- Q. 이걸 어떻게 정의하지? → 4주차 SQL

필기 ✍️

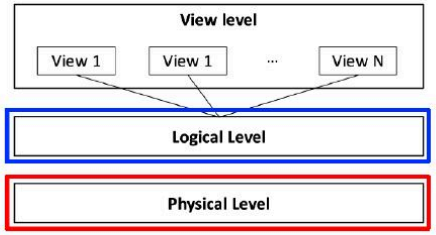
★ 시험 meta-data는 DBMS catalog에 저장 — 빈칸 단골

💡 이해 메타데이터 = '데이터에 대한 데이터'. 책장(DB)과 도서 목록 카드(catalog)

2) Data Abstraction

❑ **Insulation between Programs and Data**

- ■ A database system provides a *conceptual representation (logical level)* of data
  - ❑ E.g., objects, their properties, and their interrelationship
- ■ It does not include the complex structures of a database (*physical level*)
- ■ *A change of data structure does not affect programs* (**program-data independence**)



Data Item Name	Starting Position in Record	Length in Characters (bytes)
Name	1	30
Student_number	31	4
Class	35	1
Major	36	4

File storage format for a STUDENT record

8

2) 데이터 추상화 (Data Abstraction)

- **프로그램과 데이터 사이의 절연**
  - DB 시스템은 데이터의 **개념적 표현(logical level)**을 준다
    - 예: 객체, 그 속성, 서로의 관계
  - DB의 복잡한 저장 구조(**physical level**)는 드러내지 않는다
  - **자료 구조가 바뀌어도 프로그램은 영향받지 않는다** (**program-data independence**)
- 그림: View level → Logical level → Physical level / STUDENT 레코드의 파일 저장 형식(시작 위치·길이)

필기 ✍️

★ 시험 결과 = program-data independence (프로그램-데이터 독립성)

💡 이해 운전자는 엔진 구조를 몰라도 운전한다 = 프로그램은 저장 형식을 몰라도 된다

3) Support of Multiple Views

❑ **Different Users May Require Different Views of the Data**

■ View: **virtual data** derived from the database, but not explicitly stored or derived

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

PREREQUISITE

Course_number	Prerequisite_number
CS3380	CS3320
CS3380	MATH2410
CS3320	CS1310

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

University Database

TRANSCRIPT

Student_name	Student_transcript				
	Course_number	Grade	Semester	Year	Section_id
Smith	CS1310	C	Fall	08	119
	MATH2410	B	Fall	08	112
Brown	MATH2410	A	Fall	07	85
	CS1310	A	Fall	07	92
	CS3320	B	Spring	08	102
	CS3380	A	Fall	08	135

View 1: accessing/printing the transcript of each student

COURSE_PREREQUISITES

Course_name	Course_number	Prerequisites
Database	CS3380	CS3320
		MATH2410
Data Structures	CS3320	CS1310

View 2: checking the prerequisites of each course

9

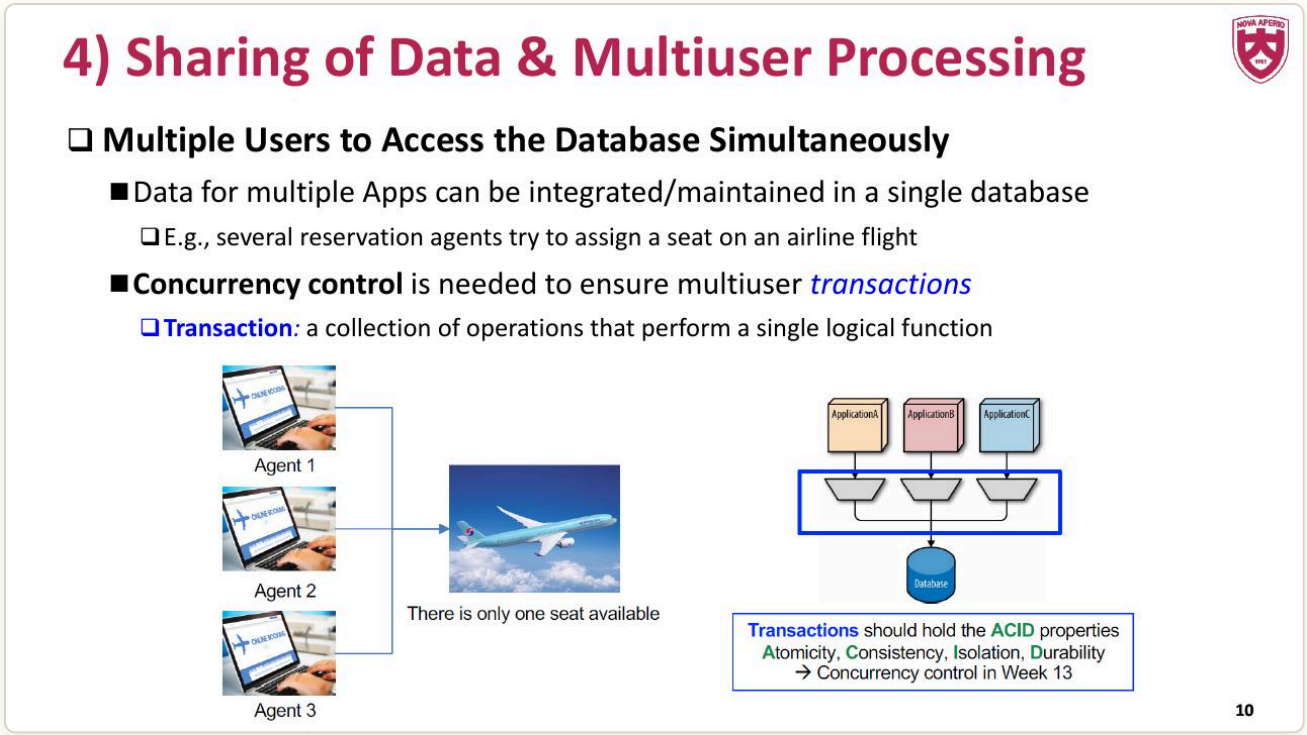
3) 다중 뷰 지원 (Multiple Views)

- 사용자마다 **다른 뷰(view)**가 필요할 수 있다
  - **View**: DB에서 끌어낸 **가상 데이터** — 따로 저장해 두지 않는다
- 예: University DB(SECTION, GRADE_REPORT, STUDENT, COURSE, PREREQUISITE)에서
  - View 1: 학생별 성적표(TRANSCRIPT)
  - View 2: 과목별 선수과목(COURSE_PREREQUISITES)

필기 ✍️

★ 시험 view = virtual data, not explicitly stored

⚠ 주의 뷰는 복사본 표가 아니다! 원래 표에서 그때그때 만들어 보여 줌 (5강 view와 연결)



4) 데이터 공유와 다중 사용자 처리

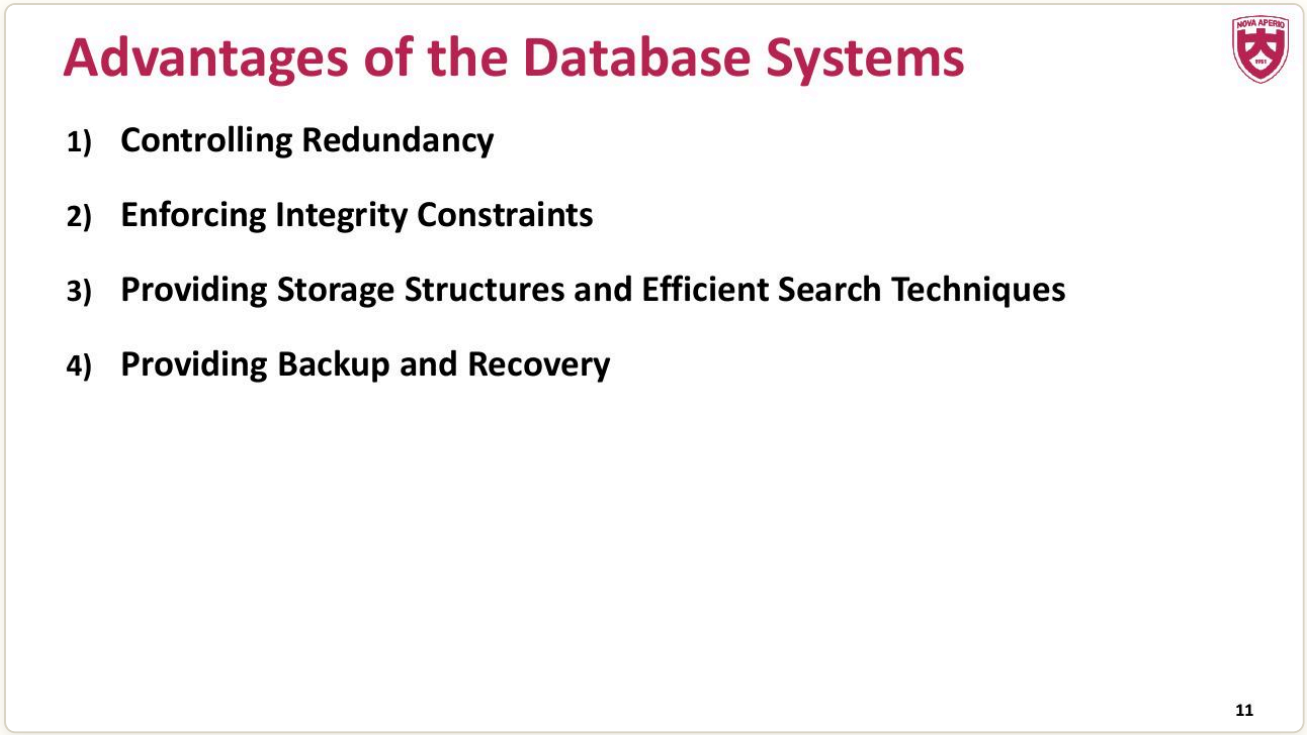
- 여러 사용자가 **동시에** DB에 접근
  - 여러 앱의 데이터를 **한 DB에 통합·유지**할 수 있다
    - 예: 여러 예약 직원이 한 비행기의 같은 좌석을 동시에 배정하려 함
  - 다중 사용자 트랜잭션을 보장하려면 **concurrency control(동시성 제어)**가 필요
    - **Transaction**: 하나의 논리적 기능을 하는 연산들의 모음
- 그림 상자: 트랜잭션은 **ACID**를 지켜야 함 — Atomicity, Consistency, Isolation, Durability → 13주차

필기 ✍️

★ 시험 Transaction 정의 영어: a collection of operations that perform a single logical function

✏ 예.풀이 남은 좌석 1개에 직원 3명이 동시에 예약 → 동시성 제어가 없으면 중복 배정

💡 이해 ACID는 슬라이드 그림 속에 있음. 자세한 건 13주차(중간 범위 밖)



DB 시스템의 장점

- 1) **Controlling Redundancy** — 중복 통제
- 2) **Enforcing Integrity Constraints** — 무결성 제약 강제
- 3) **Storage Structures & Efficient Search** — 저장 구조와 효율적 검색
- 4) **Backup and Recovery** — 백업과 복구

필기 ✍️

★ 시험 장점 4개 + p.20 요약에 하나 더:
Restricting Unauthorized Access(허가 안 된 접근 제한)

1) Controlling Redundancy

❑ What is Data Redundancy?

■ Storing the same data in multiple times in different locations

❑ Problems

■ P1: Have to perform a single logical update multiple times (***duplication of effort***)

■ P2: Storage space is ***wasted*** (e.g., Hard disk or SSD)

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

What if Name of Student or Course_name is updated?

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Smith	112	MATH2410	B
17	Smith	119	CS1310	C
8	Brown	85	MATH2410	A
8	Brown	92	CS1310	A
8	Brown	102	CS3320	B
8	Brown	135	CS3380	A

12

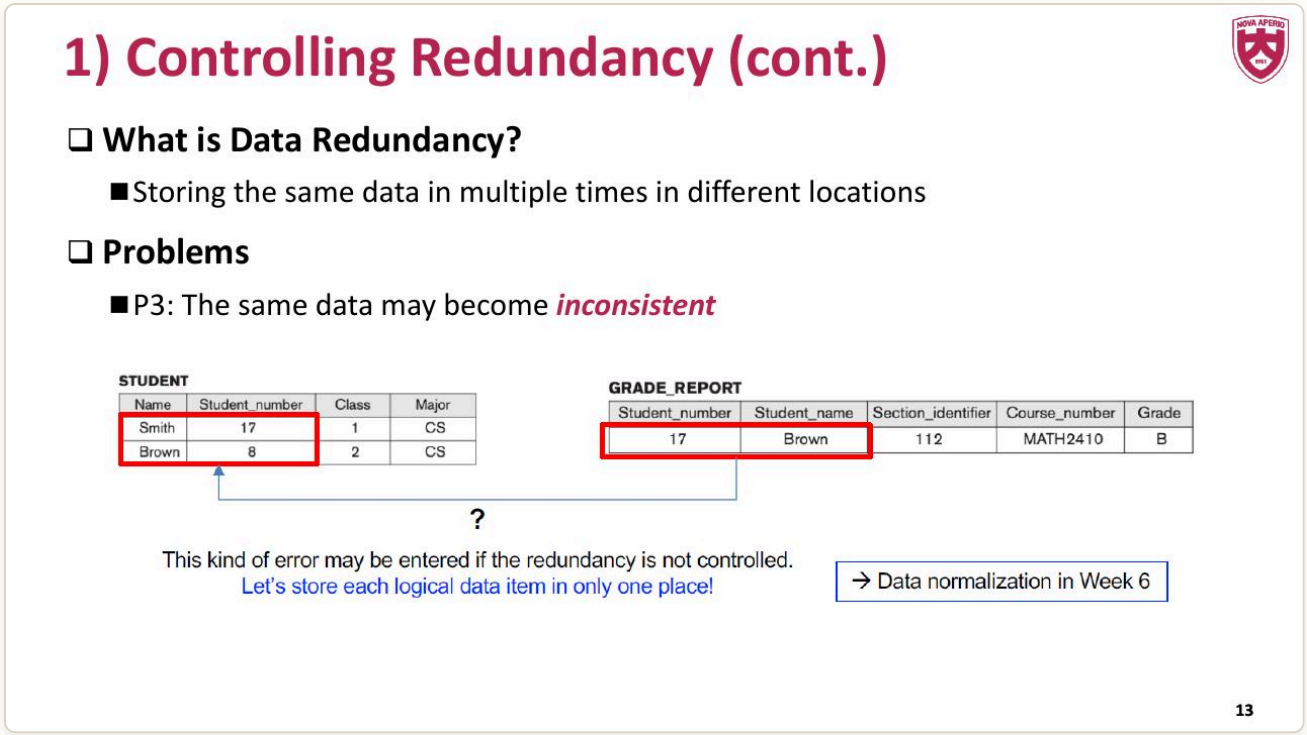
1) 중복 통제

- **Data Redundancy**: 같은 데이터를 여러 곳에 여러 번 저장하는 것
- 문제점
  - **P1**: 논리적으로 한 번 고칠 것을 여러 번 고쳐야 함 (**duplication of effort**)
  - **P2**: 저장 공간 **낭비** (HDD·SSD)
- 그림: GRADE_REPORT에 Student_name·Course_number까지 넣으면 → 학생 이름·과목명이 바뀔 때?

필기 ✍️

★ 시험 P1 duplication of effort · P2 storage wasted · P3 inconsistent (다음 쪽)

— 3개 세트



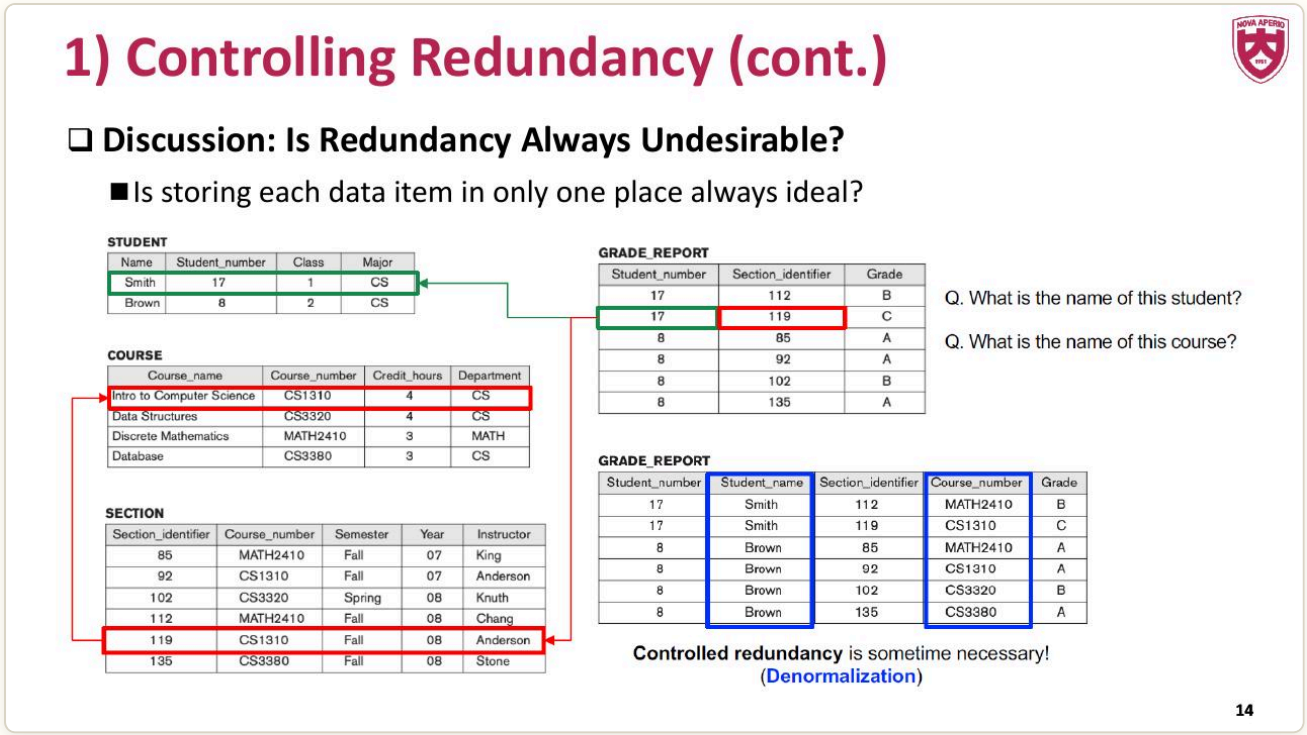
1) 중복 통제 (계속)

- **P3**: 같은 데이터가 **서로 달라질(inconsistent)** 수 있다
- 그림: STUDENT에선 17번 = Smith인데 GRADE_REPORT에는 17번 = Brown
  - 중복을 통제하지 않으면 이런 오류가 들어온다
  - → 논리적 데이터 하나는 **한 곳에만** 저장하자!
- → 6주차 데이터 정규화(normalization)

필기 ✍️

✏ 예.풀이 한쪽만 고치면 같은 학번에 이름 두 개 = 불일치(inconsistency)

⚠ 주의 6주차 정규화 자료는 아직 못 받음 — 개념만 연결해 두기



1) Controlling Redundancy (cont.)

❑ Discussion: Is Redundancy Always Undesirable?

■ Is storing each data item in only one place always ideal?

STUDENT

Name	Student_number	Class	Major
Smith	17	1	CS
Brown	8	2	CS

COURSE

Course_name	Course_number	Credit_hours	Department
Intro to Computer Science	CS1310	4	CS
Data Structures	CS3320	4	CS
Discrete Mathematics	MATH2410	3	MATH
Database	CS3380	3	CS

SECTION

Section_identifier	Course_number	Semester	Year	Instructor
85	MATH2410	Fall	07	King
92	CS1310	Fall	07	Anderson
102	CS3320	Spring	08	Knuth
112	MATH2410	Fall	08	Chang
119	CS1310	Fall	08	Anderson
135	CS3380	Fall	08	Stone

GRADE_REPORT

Student_number	Section_identifier	Grade
17	112	B
17	119	C
8	85	A
8	92	A
8	102	B
8	135	A

Q. What is the name of this student?

Q. What is the name of this course?

GRADE_REPORT

Student_number	Student_name	Section_identifier	Course_number	Grade
17	Smith	112	MATH2410	B
17	Smith	119	CS1310	C
8	Brown	85	MATH2410	A
8	Brown	92	CS1310	A
8	Brown	102	CS3320	B
8	Brown	135	CS3380	A

Controlled redundancy is sometime necessary!
(Denormalization)

14

1) 중복 통제 (계속) — 토론

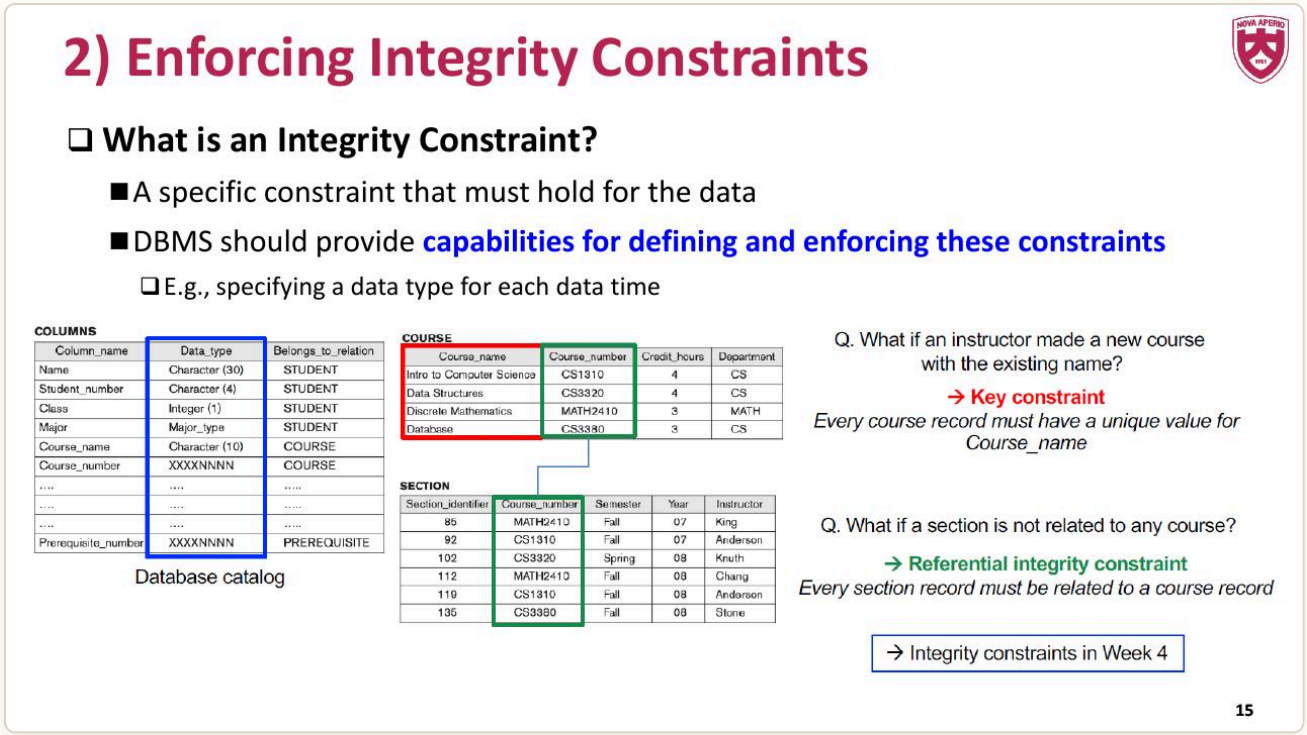
- **중복은 언제나 나쁠까?**
  - 데이터 하나를 꼭 한 곳에만 두는 게 항상 이상적일까?
- 그림: 성적표에서 학생 이름·과목명을 보려면 매번 다른 표를 찾아가야 함
  - 아래처럼 Student_name, Course_number를 같이 둔 GRADE_REPORT
  - **Controlled redundancy is sometime necessary! (Denormalization)**

필기 ✍️

★ 시험 답: 아니다 — 통제된 중복(controlled redundancy)은 필요할 때가 있다 = Denormalization

⚠ 주의 이 문장은 슬라이드 '그림 속 글'에만 있다. 놓치기 쉬움!

💡 이해 자주 보는 조합은 같이 저장해 두면 조회가 빠름 (대신 갱신 때 신경 써야)



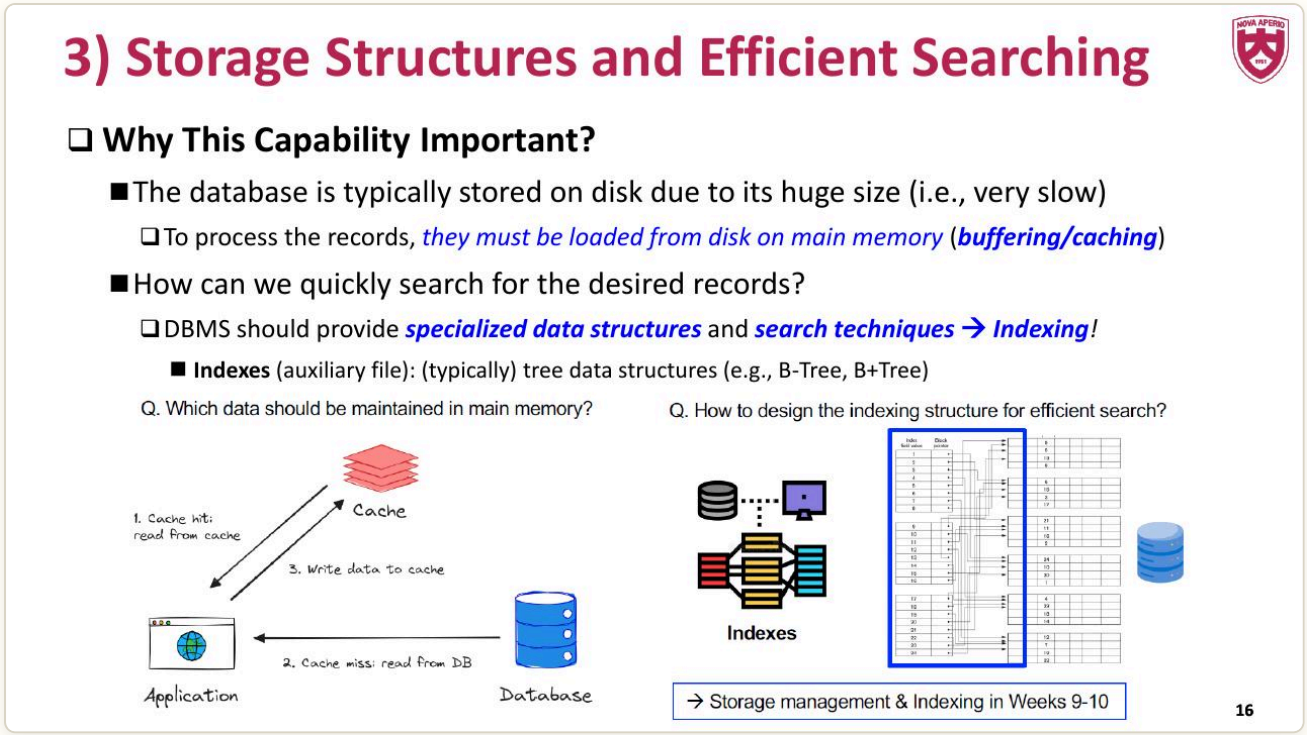
2) 무결성 제약 강제

- **Integrity Constraint**: 데이터가 반드시 지켜야 하는 제약
  - DBMS는 이 제약을 **정의하고 강제하는 기능**을 제공해야 한다
    - 예: 데이터 항목마다 자료형 지정
- 그림 Q1. 강사가 이미 있는 과목명으로 새 과목을 만들면?
  - → **Key constraint** — 모든 과목 레코드는 Course_name이 서로 달라야 함
- 그림 Q2. 어떤 과목과도 연결 안 된 분반이 있으면?
  - → **Referential integrity constraint** — 모든 분반은 과목 레코드와 연결돼야 함
- → 4주차 무결성 제약

필기 ✍️

★ 시험 예시 2개 짝: 같은 이름 과목 = Key constraint / 과목 없는 분반 = Referential integrity

💡 이해 슬라이드 'data time'은 'data item'의 오타



3) 저장 구조와 효율적 검색

- 왜 중요한가?
  - DB는 크기가 커서 보통 **디스크**에 저장됨 (아주 느림)
    - 처리하려면 디스크에서 **메인 메모리로 읽어 와야** 함 (**buffering/caching**)
  - 원하는 레코드를 어떻게 빨리 찾나?
    - DBMS는 특별한 자료 구조와 검색 기법을 줘야 함 → **Indexing!**
      - **Index (auxiliary file, 보조 파일)**: 보통 트리 구조 (B-Tree, B+Tree)
- 그림: 캐시 hit/miss, 인덱스 구조 → 9~10주차

필기 ✍️

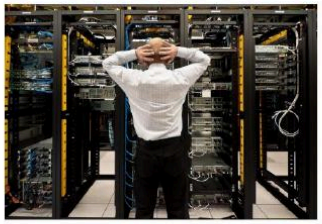
★ 시험 Index = auxiliary file, 보통 tree (B-Tree, B+Tree)

💡 이해 책 뒤의 '찾아보기'와 같다 — 처음부터 다 안 읽고 쪽 번호로 바로 감

4) Providing Backup and Recovery

❑ **What if the System Fails during a Complex Update Transactions?**

- A DBMS should provide **facilities for recovering** from hardware or software failures
- Backup
  - ❑ Making a copy of a database inside DBMS in normal processing for future recovery
- Recovery
  - ❑ Making a database recovered to the latest state it was before the failures



Hardware or software failures can occur unexpectedly

Q. How to restore the DB to the original state?



Backup and Recovery

→ Recovery techniques in Week 13

17

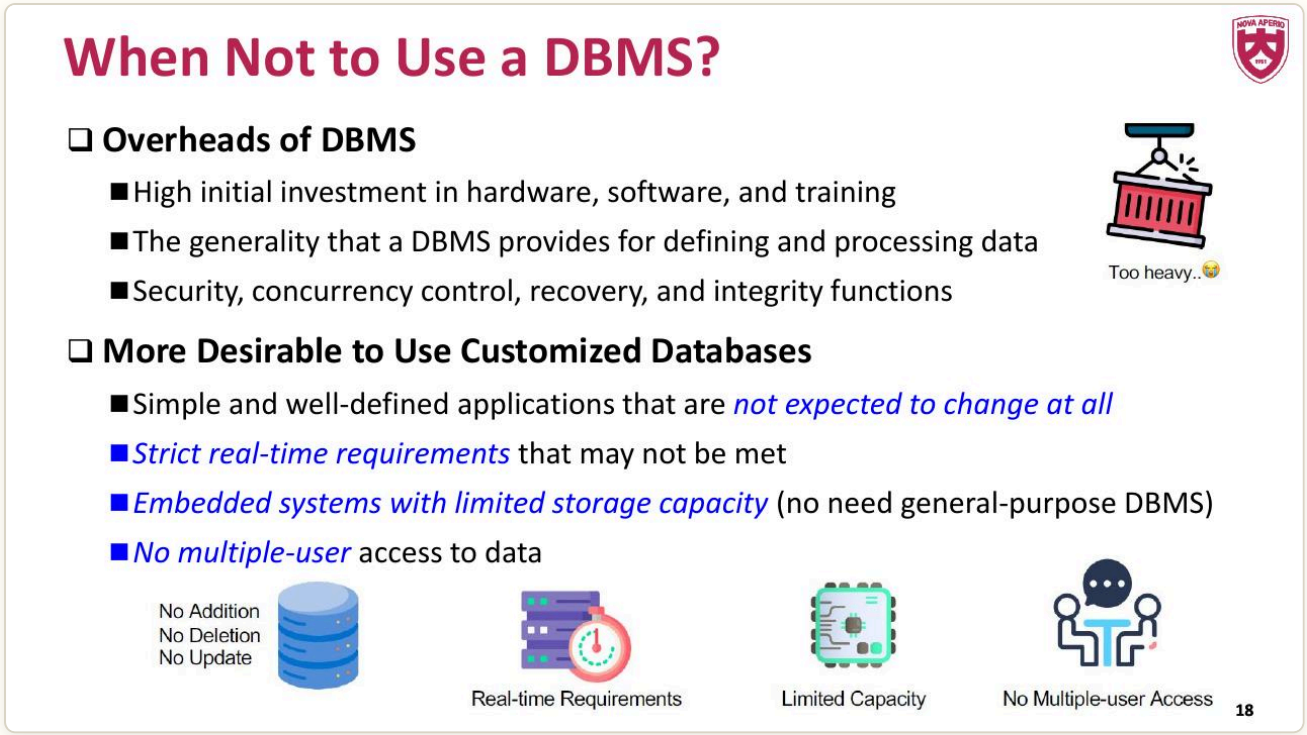
4) 백업과 복구

- 복잡한 갱신 트랜잭션 도중 시스템이 멈추면?
  - DBMS는 HW·SW 고장에서 **회복하는 기능**을 줘야 함
  - **Backup**: 평소에 DB 복사본을 만들어 두는 것 (나중 복구용)
  - **Recovery**: 고장 **직전의 최신 상태**로 DB를 되돌리는 것
- → 13주차 복구 기법

필기 ✍️

⚠ 주의 Backup(미리 복사) ≠ Recovery(고장 후 되돌리기). 정의 바꿔 내면 틀림

★ 시험 Recovery = the latest state it was before the failures



DBMS를 쓰지 않는 게 나을 때

- **DBMS의 부담(Overheads)**
  - HW·SW·교육에 드는 큰 초기 투자
  - DBMS가 주는 범용성(generality) — 정의·처리 기능 전반
  - 보안·동시성 제어·복구·무결성 기능의 비용
- **맞춤형(customized) DB가 나은 경우**
  - 단순하고 잘 정의돼 **전혀 바뀌지 않을** 앱 (추가·삭제·갱신 없음)
  - **엄격한 실시간** 요구 — 못 맞출 수 있음
  - 저장 용량이 작은 **임베디드 시스템**
  - **여러 사용자가 접근하지 않음**

필기 ✍️

★ 시험 부담 3개 + 맞춤형이 나은 경우 4개 — 개수까지 외우기

💡 이해 자전거로 마트 가는데 트럭 살 필요 없다 🚲

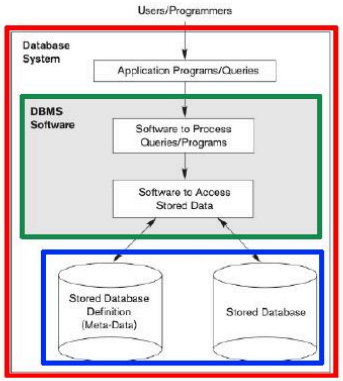
Summary: Database System Concepts

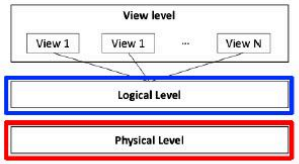
❑ **Introduction to Database**

- Database, DBMS, and Database System

❑ **Characteristics of DB Systems**

- Self-describing nature
- Data abstraction
- Support of multiple views of the data
- Sharing of data and multiuser transaction processing





TRANSCRIPT

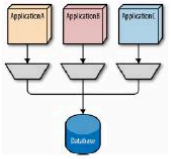
Student_name	Student_transcript				
	Course_number	Grade	Semester	Year	Section_id
Smith	CS1310	C	Fall	08	119
	MATH2410	B	Fall	08	112
Brown	MATH2410	A	Fall	07	85
	CS1310	A	Fall	07	92
	CS3320	B	Spring	08	102
	CS3380	A	Fall	08	135

View 1

COURSE_PREREQUISITES

Course_name	Course_number	Prerequisites
Database	CS3380	CS3320
		MATH2410
Data Structures	CS3320	CS1310

View 2



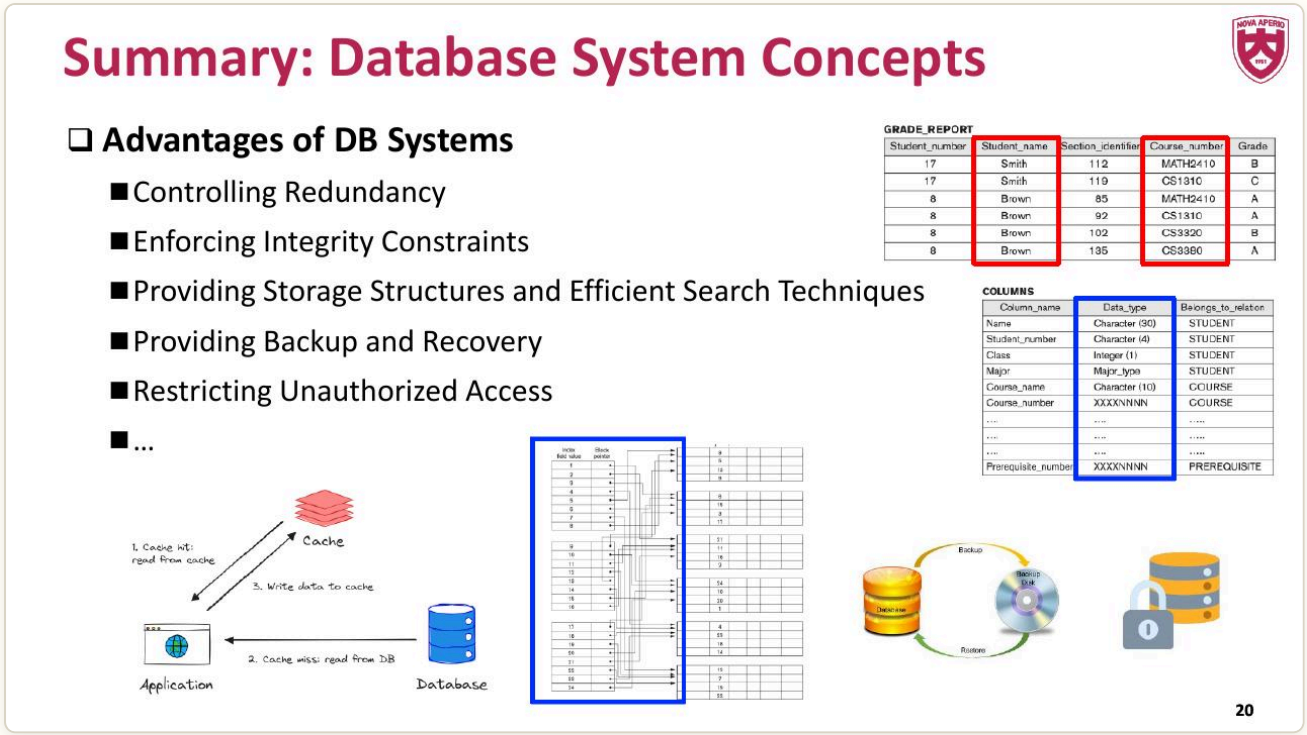
19

요약: DB 시스템 개념

- DB 소개 — Database, DBMS, Database System
- DB 시스템의 특성
  - Self-describing nature
  - Data abstraction
  - Support of multiple views of the data
  - Sharing of data and multiuser transaction processing

필기 ✍️

★ 시험 특성 4개 다시 한 번 (영어로 써 보기)



요약: DB 시스템 개념 (계속)

- DB 시스템의 장점
  - Controlling Redundancy
  - Enforcing Integrity Constraints
  - Providing Storage Structures and Efficient Search Techniques
  - Providing Backup and Recovery
  - **Restricting Unauthorized Access** (허가 안 된 접근 제한)
  - ...

필기 ✍️

⚠ 주의 '허가 안 된 접근 제한'은 본문(p.11)엔 없고 요약에만 있음 — 장점으로 인정

Outline

❑ **Introduction to DB Systems**
- Characteristics and advantages

❑ **DB system Architecture**
- Conceptual, representational, and physical data models
- Three-schema architecture and data independence

❑ **Entity-Relationship (ER) Model**
- Entity types and attributes
- Initial conceptual design
- Relationship types, constraints, and attributes
- Refining conceptual design

21

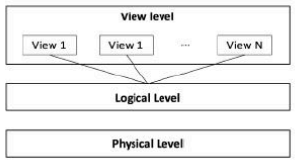
목차 (Outline)

- 이제 두 번째 덩어리: **DB 시스템 구조**
  - 개념·표현·물리 데이터 모델
  - 3단계 스키마 구조와 데이터 독립성

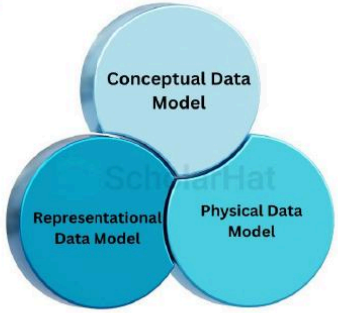
필기 ✍️
— 넘어가도 되는 쪽

What is a Data Model?

❑ **Data Abstraction**
- The highlighting of the essential features for understanding of data
  - Thus, different users can perceive data at their preferred level of detail
- The suppression of details of data organization and storage



❑ **Data Model**
- A collection of concepts that ***describe the structure of a database***
- Provides the necessary "**Tools**" to achieve data abstraction
  - E.g., data types, relations, and constraints that apply to the data
  - E.g., basic operations for specifying retrievals and updates on the DB
- Three categories (*How to describe the DB structure*)
  - Conceptual data models (high-level)
  - Representational data models (between these two levels)
  - Physical data models (low-level)



22

데이터 모델이란?

- **Data Abstraction (데이터 추상화)**
  - 데이터 이해에 필요한 핵심을 **드러내기(highlighting)**
    - 그래서 사용자마다 원하는 수준으로 데이터를 볼 수 있다
  - 데이터 조직·저장의 세부는 **감추기(suppression)**
- **Data Model**
  - **DB의 구조를 기술하는 개념들의 모음**
  - 데이터 추상화를 이루는 '**도구(Tools)**'
    - 예: 자료형, 관계, 데이터에 걸리는 제약
    - 예: 검색·갱신을 지정하는 기본 연산
  - 세 범주 (DB 구조를 어떻게 기술하나)
    - **Conceptual** (high-level) / **Representational** (중간) / **Physical** (low-level)

필기 ✍️

★ 시험 Data model = a collection of concepts that describe the structure of a database

★ 시험 추상화 = highlighting(드러내기) + suppression(감추기) 두 단어 세트

Conceptual Data Models

❑ **Providing Concepts Close to *the Way Users Perceive Data***

- ■ **Entity**: a real-world object or concept
- ■ **Attribute**: a property of interest that describes an entity
- ■ **Relationship**: an association among two or more entities

❑ **Example: Company Database**



How can we represent the data about this "Company"?

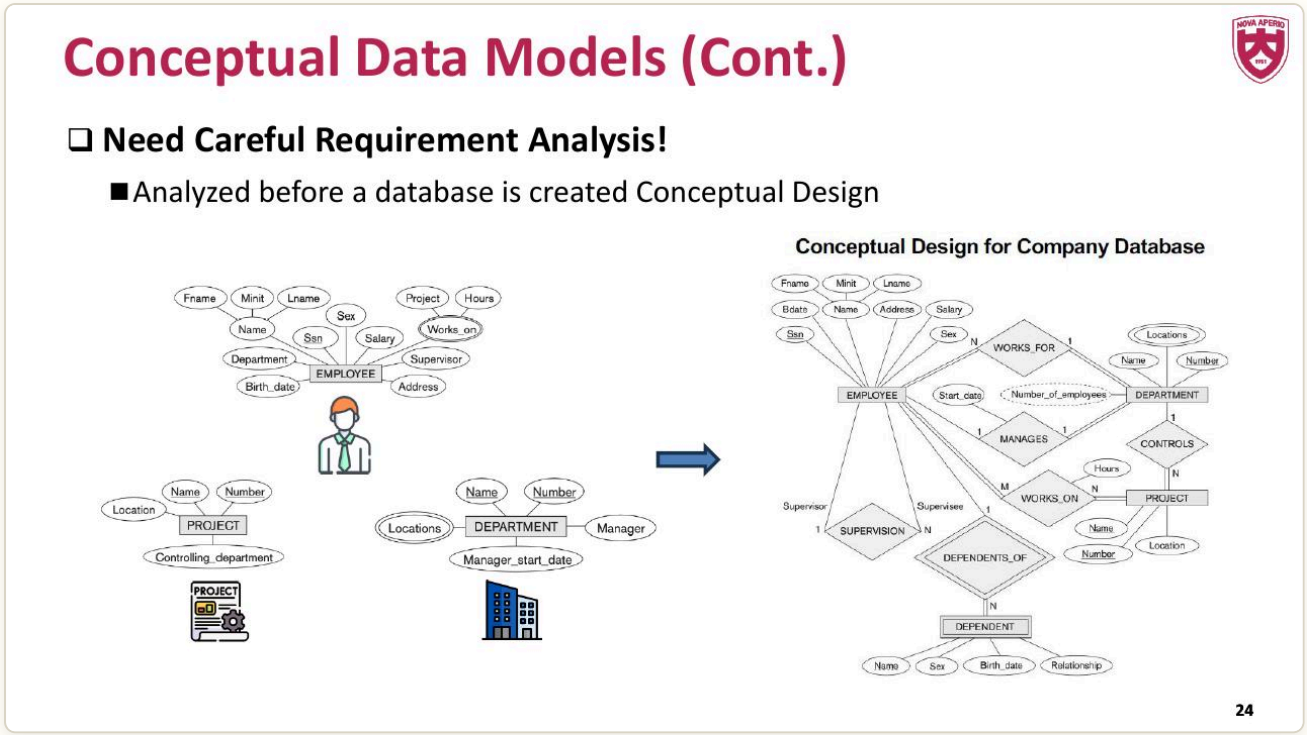
23

개념적 데이터 모델 (Conceptual)

- **사용자가 데이터를 바라보는 방식에 가까운** 개념을 준다
  - **Entity**: 현실 세계의 객체나 개념
  - **Attribute**: 엔티티를 설명하는 관심 있는 성질
  - **Relationship**: 둘 이상의 엔티티 사이의 연관
- 예: 회사 DB — 부서 A·B·C, 직원(이름·급여·주소·부서…), 프로젝트(이름·관리 부서…)
  - 이 '회사' 데이터를 어떻게 표현할까?

필기 ✍️

★ 시험 Entity / Attribute / Relationship 정의 한 줄씩 영어로

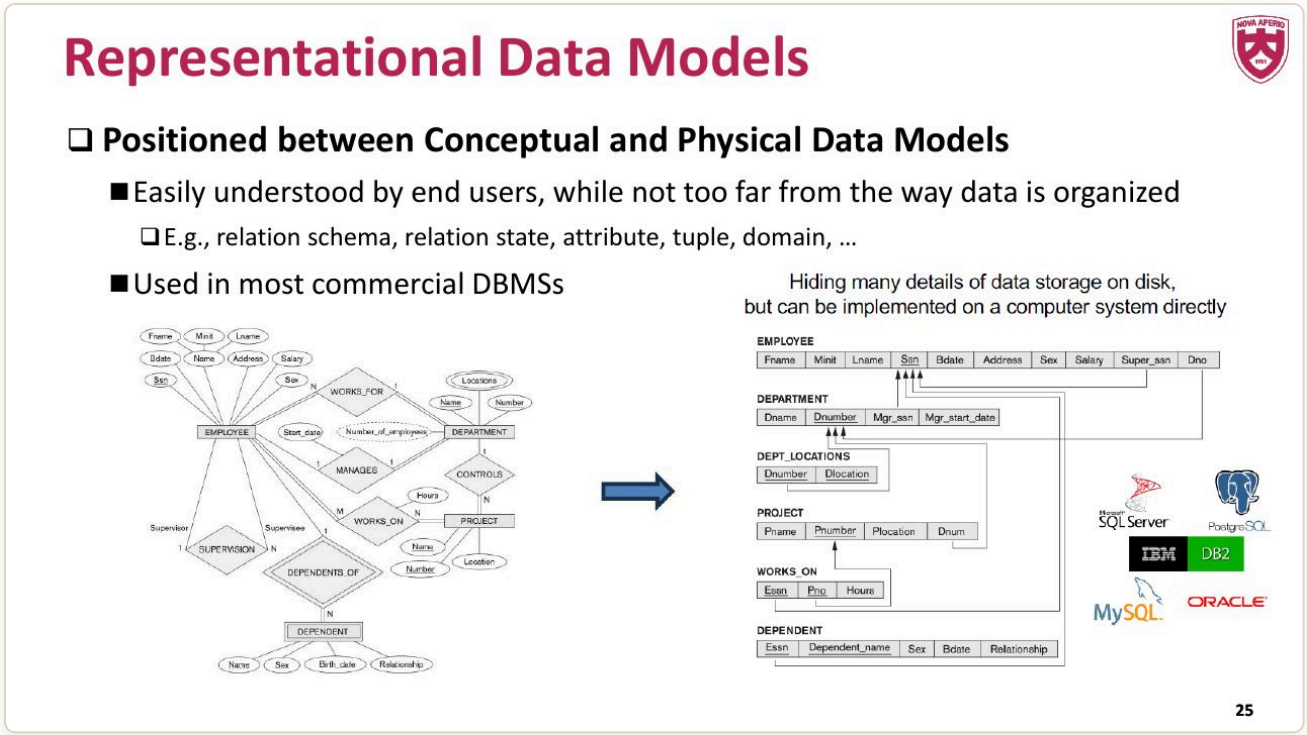


개념적 데이터 모델 (계속)

- **꼼꼼한 요구사항 분석이 필요!**
  - DB를 만들기 전에 분석 → **개념 설계(Conceptual Design)**
- 그림: 엔티티별 속성 그림 → 회사 DB의 개념 설계(ER 다이어그램)

필기 ✍️

💡 이해 오른쪽 ER 다이어그램이 이 강의 끝(p.79)의 완성본 — 지금은 눈에 익혀 두기만



표현 데이터 모델 (Representational)

- 개념 모델과 물리 모델 **사이**에 위치
  - 최종 사용자도 이해하기 쉽고, 실제 저장 방식과도 너무 멀지 않음
    - 예: relation schema, relation state, attribute, tuple, domain …
  - **대부분의 상용 DBMS**가 사용
- 그림: ER 다이어그램 → 관계형 표(EMPLOYEE, DEPARTMENT …)
  - 디스크 저장 세부는 감추지만 컴퓨터에 바로 구현할 수 있음

필기 ✍️

✏ 예.풀이 관계형 모델(표·튜플·도메인)이 대표 예

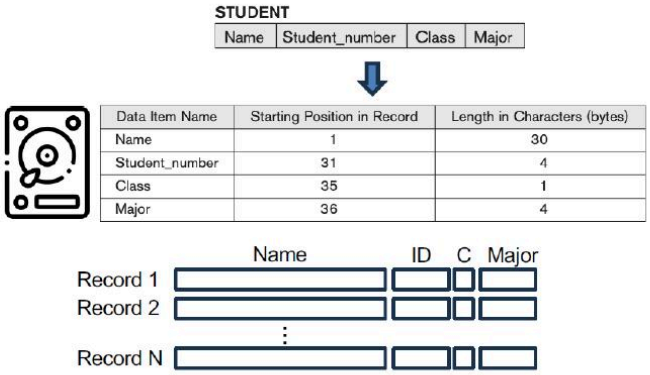
— 3강 내용

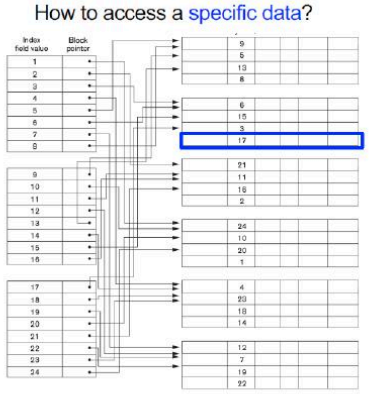
★ 시험 used in most commercial DBMSs

Physical Data Models

❑ **Describing *How Data is Stored in a Computer System***

- Representing information such as *record formats*, *orderings*, and *access paths*
- The provided concepts are meant for computer specialists, not for end users





26

물리 데이터 모델 (Physical)

- **데이터가 컴퓨터에 어떻게 저장되는지** 기술
  - 레코드 형식(record formats), 순서(orderings), 접근 경로(access paths)
  - 최종 사용자가 아니라 **컴퓨터 전문가**용 개념
- 그림: STUDENT 레코드의 바이트 위치·길이, 인덱스로 특정 데이터 찾기

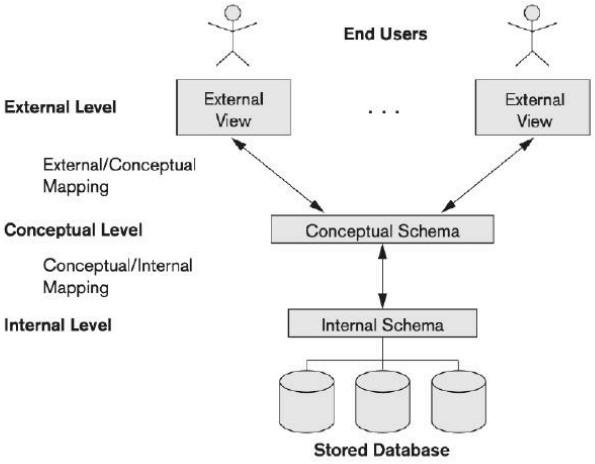
필기 ✍️

★ 시험 키워드 3개: record formats · orderings · access paths

Three-Schema Architecture

❑ **Goal:** ***To Separate the User App from the Physical DB***

■ For the independence between a user program and data



27

3단계 스키마 구조 (Three-Schema Architecture)

- 목표: **사용자 앱을 물리 DB와 떼어 놓기**
  - 사용자 프로그램과 데이터 사이의 **독립성**을 위해
- 그림: External Level(외부 뷰들) → Conceptual Level(개념 스키마) → Internal Level(내부 스키마) → 저장된 DB
  - 층 사이에 External/Conceptual Mapping, Conceptual/Internal Mapping

필기 ✍️

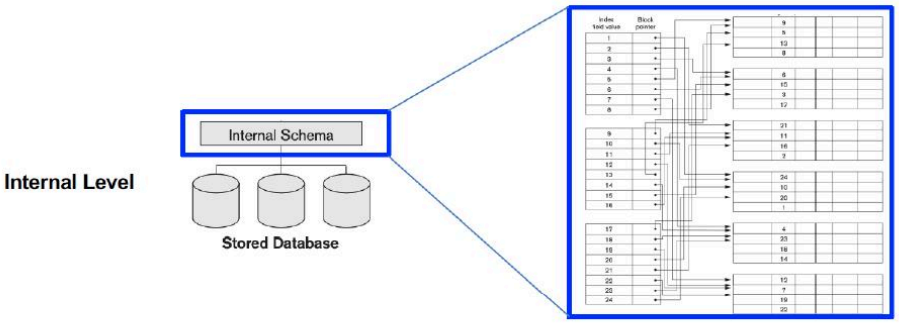
★ 시험 그림을 직접 그릴 수 있게: 3층 + 매핑 2개

💡 이해 위로 갈수록 사용자 쪽, 아래로 갈수록 디스크 쪽

Three-Schema Architecture: Internal Level

❑ **Internal Level**

- **Internal schema**: to describes the physical storage structure of the database
- Using a **physical data model** to describe the details of data storage and access paths for the database



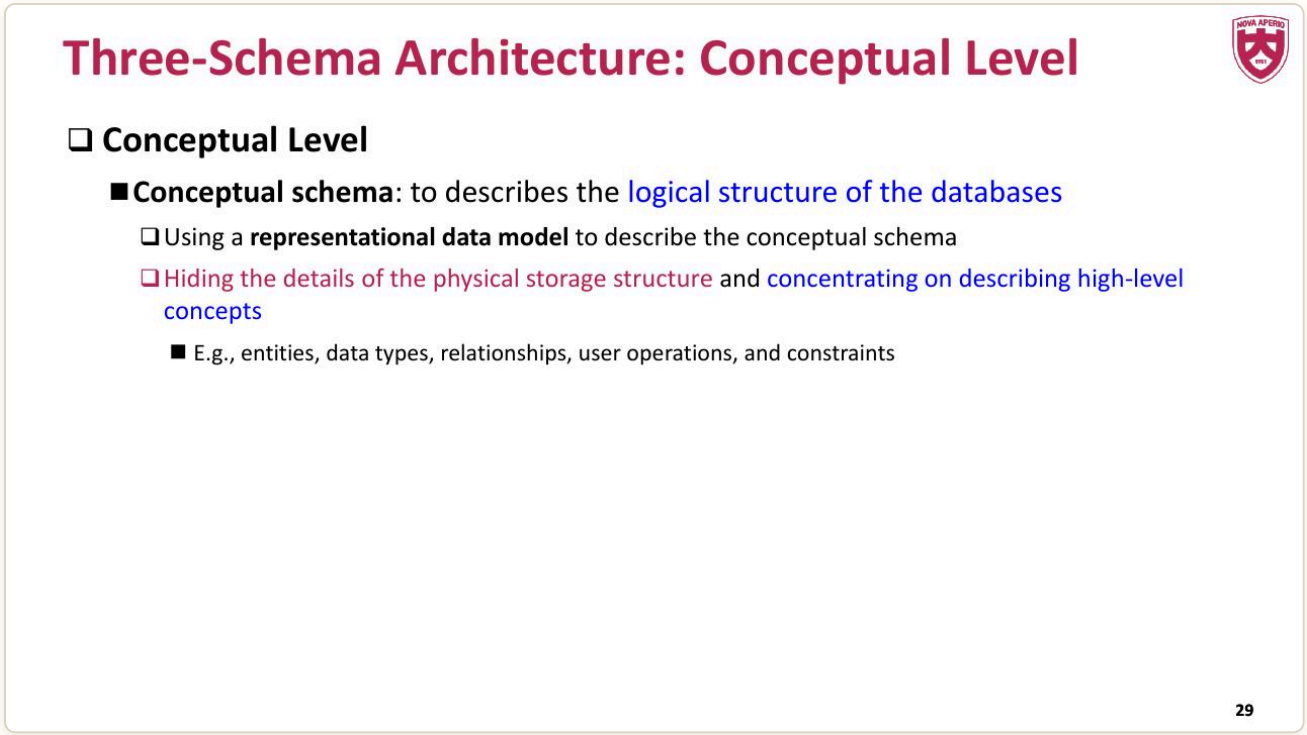
28

3단계 스키마: 내부 단계 (Internal)

- **Internal schema**: DB의 **물리적 저장 구조**를 기술
  - **물리 데이터 모델**로 저장 세부와 **접근 경로**를 기술

필기 ✍️

★ 시험 Internal ↔ Physical data model 짝



3단계 스키마: 개념 단계 (Conceptual)

- **Conceptual schema**: DB 전체의 **논리적 구조**를 기술
  - **표현 데이터 모델(representational)**로 기술
  - 물리 저장 구조는 **감추고**, 높은 수준의 개념에 집중
    - 예: 엔티티, 자료형, 관계, 사용자 연산, 제약

필기 ✍️

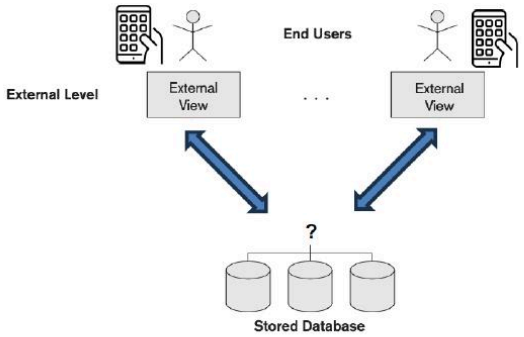
⚠ 주의 개념 '스키마'는 representational 모델로 기술! (conceptual data model과 이름이 비슷해 함정)

Three-Schema Architecture: External Level

❑ **External (View) Level**

■ **External Schema**: to describes the part of the database structure that a particular user group is interested in

❑ Hiding the rest of the database from that user group



Each user group refers to a database
in the way as described in its external (logical) schema

However, the actual data is stored at the physical level only.
Thus, how can the DBMS support each user request?

30

3단계 스키마: 외부 단계 (External / View)

- **External schema**: 특정 **사용자 그룹이 관심 있는 부분**만 기술
  - 나머지는 그 그룹에게 **감춘다**
- 그림 설명: 각 그룹은 자기 외부 스키마대로 DB를 본다
  - 하지만 실제 데이터는 물리 단계에만 있다 — DBMS는 요청을 어떻게 처리할까?

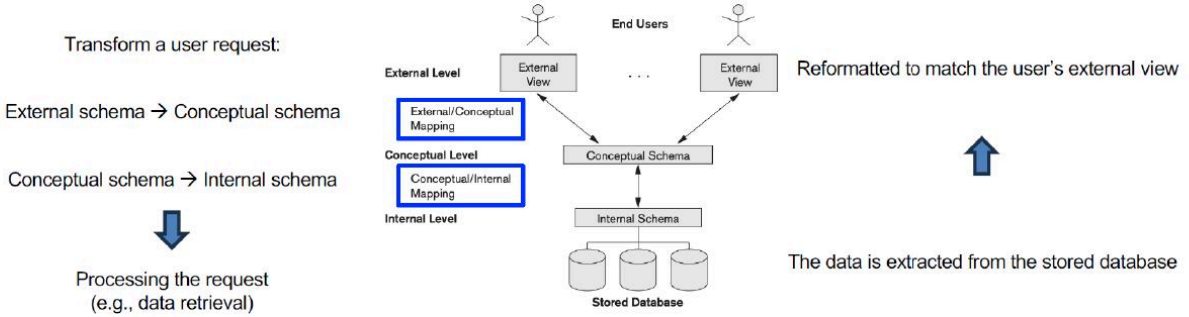
필기 ✍️

💡 이해 → 답은 다음 쪽 'Mapping'

Three-Schema Architecture: Mapping

❑ **Transforming Requests and Results between Levels**

- Actual data is stored at the physical level only
- The DBMS must transform a request specified on a high-level schema into a request against a lower-level schema



Mapping information is also stored in the catalog

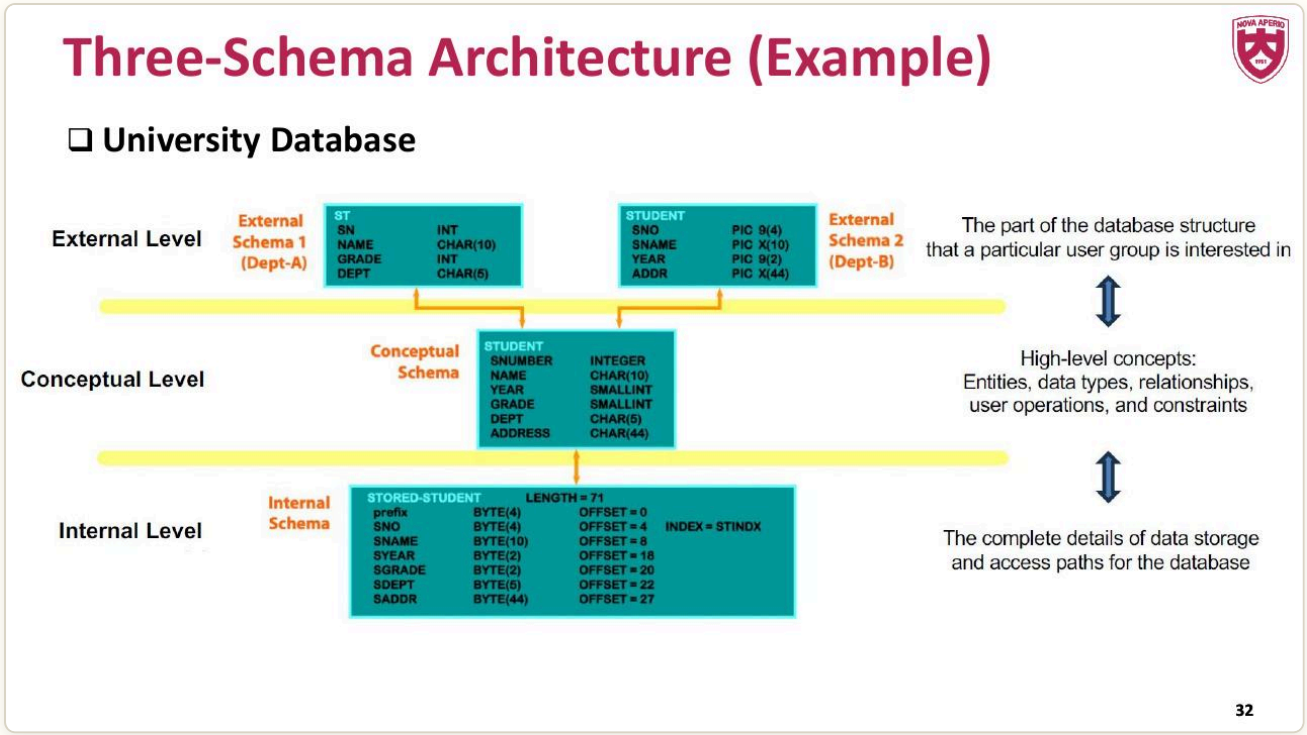
31

3단계 스키마: 매핑 (Mapping)

- **단계 사이에서 요청과 결과를 변환**
  - 실제 데이터는 **물리 단계에만** 저장
  - DBMS는 상위 스키마로 표현된 요청을 **하위 스키마 요청으로 바꿔야** 함
- 그림: 요청 — External → Conceptual → Internal 로 변환해 처리
  - 결과 — 저장된 DB에서 꺼내 사용자의 외부 뷰 모양으로 다시 맞춤
  - **매핑 정보도 catalog에 저장**

필기 ✍️

★ 시험 Mapping information is also stored in the catalog



3단계 스키마 (예: 대학 DB)

- External: 학과 A용 ST(SN, NAME, GRADE, DEPT), 학과 B용 STUDENT(SNO, SNAME, YEAR, ADDR)
- Conceptual: STUDENT(SNUMBER, NAME, YEAR, GRADE, DEPT, ADDRESS) + 자료형
- Internal: STORED-STUDENT — 바이트 길이·OFFSET·INDEX=STINDX
  - 외부 = 관심 있는 일부 / 개념 = 엔티티·자료형·관계·연산·제약 / 내부 = 저장·접근 경로 세부

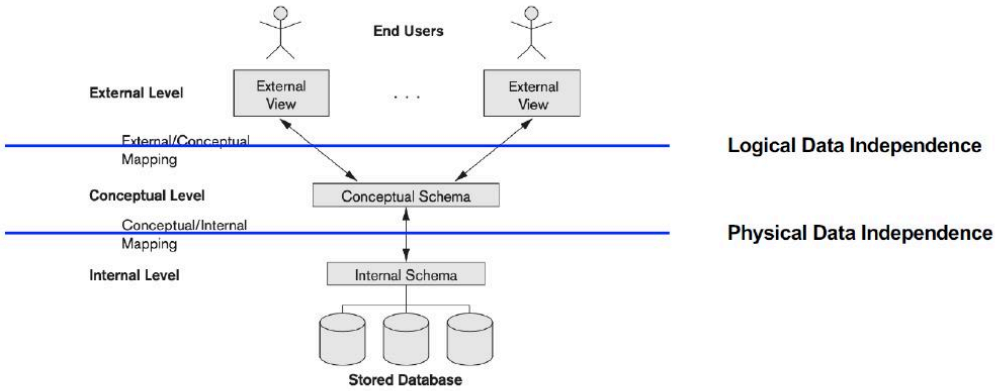
필기 ✍️

✏ 예·풀이 같은 학생 데이터가 층마다 다른 모습 — 외부는 일부 열만, 내부는 바이트 위치까지

Data Independence

❑ Definition

■ The capacity to change the schema at one level of a database system without having to change the schema at the next higher level



33

데이터 독립성 (Data Independence)

- 정의: 한 단계의 스키마를 바꿔도 **바로 위 단계 스키마는 바꿀 필요가 없는** 능력
- 그림: External/Conceptual 경계 = **Logical** Data Independence, Conceptual/Internal 경계 = **Physical** Data Independence

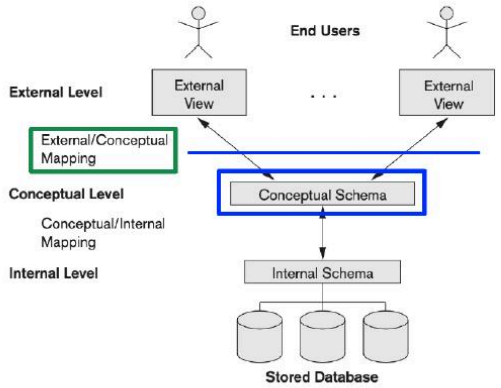
필기 ✍️

★ 시험 정의 영어: the capacity to change the schema at one level without having to change the schema at the next higher level

Logical Data Independence

❑ **The Capacity to Change the Conceptual Schema**

- Without having to change external schemas or application programs
- No need to rewrite existing application programs referring to the external schema



External schemas should not be affected,
Only the view definition and the mappings need to be updated

We may change the conceptual schema
(1) to expand the database by adding a new record type or data,
(2) change constraints,
(3) to reduce the database by removing a record type of data

34

논리적 데이터 독립성 (Logical)

- **개념 스키마**를 바꿀 수 있는 능력
  - 외부 스키마나 응용 프로그램은 **안 바꿔도** 됨
  - 외부 스키마를 쓰는 기존 프로그램을 다시 쓸 필요 없음
- 그림: 외부 스키마는 영향 없음 — **뷰 정의와 매핑만** 고치면 됨
  - 개념 스키마를 바꾸는 예: (1) 레코드 타입·데이터 추가 (2) 제약 변경 (3) 레코드 타입 제거

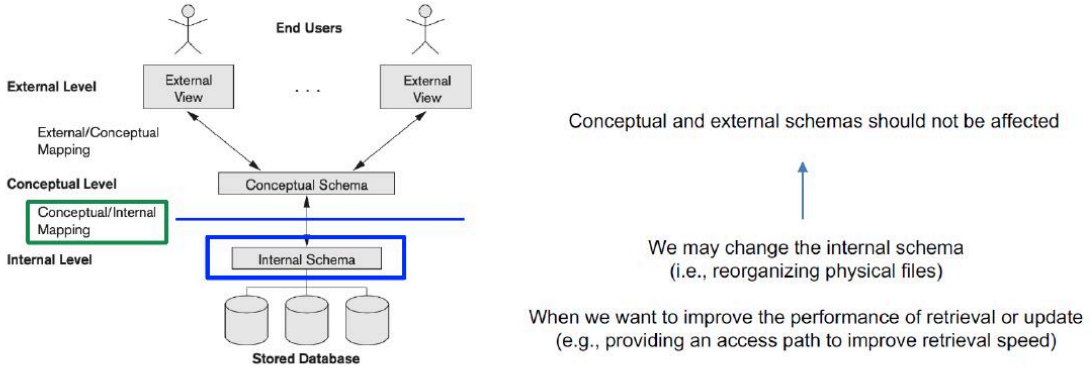
필기 ✍️

★ 시험 Logical = 개념 스키마 변경 ⟶ 외부/앱 그대로 (고치는 건 external/conceptual mapping)

Physical Data Independence

❑ **The Capacity to Change the Internal Schema**

- Without having to change the conceptual schema (external schema as well)
- No need to rewrite existing application programs referring to the external schema



35

물리적 데이터 독립성 (Physical)

- **내부 스키마**를 바꿀 수 있는 능력
  - 개념 스키마(외부 스키마도)는 **안 바꿔도** 됨
  - 기존 응용 프로그램을 다시 쓸 필요 없음
- 그림: 내부 스키마 변경 예 = 물리 파일 재구성
  - 검색·갱신 성능을 높이고 싶을 때 (예: 접근 경로 추가)

필기 ✍️

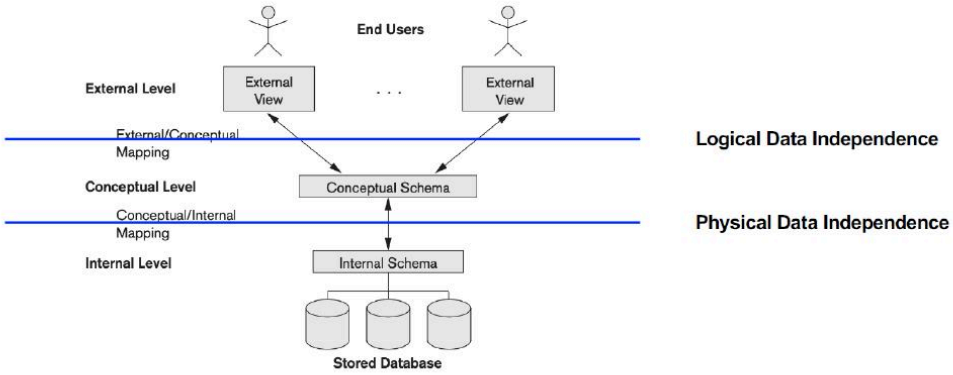
★ 시험 Physical = 내부 스키마 변경(인덱스 추가 등) ⟶ 개념·외부 그대로

⚠ 주의 Logical/Physical 반대로 쓰기 쉬움: '무엇을 바꾸나'로 구분 — 개념이면 Logical, 내부면 Physical

Advantages of Data Independence

❑ **Why Data Independence Important?**

- ■ One of the most important features of DBMSs
- ■ Significantly reduces the maintenance overhead of existing application programs



36

데이터 독립성의 장점

- 왜 중요한가?
  - DBMS의 **가장 중요한 특징 중 하나**
  - 기존 응용 프로그램의 **유지보수 부담을 크게 줄인다**

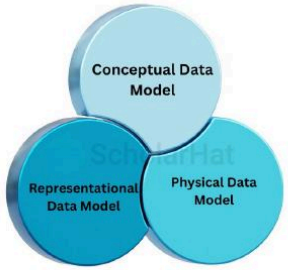
필기 ✍️

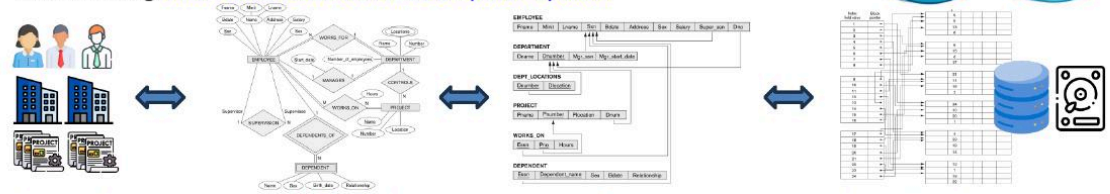
💡 이해 일반적으로 물리적 독립성보다 논리적 독립성이 이루기 더 어렵다고 알려져 있음 (참고)

Summary: Database System Architecture

❑ **Data Model: A Collection of Concepts Describing DB Structure**

- ■ Conceptual data model
  - ❑ Providing Concepts Close to *the Way Users Perceive Data*
- ■ Representational data model
  - ❑ Positioned between Conceptual and Physical Data Models
- ■ Physical data model
  - ❑ Describing *How Data is Stored in a Computer System*





Real-world Data | Conceptual Data Model | Representational Data Model | Physical Data Model

Highlighting of the essential features for understanding of data
Suppression of details of data organization and storage

37

요약: DB 시스템 구조

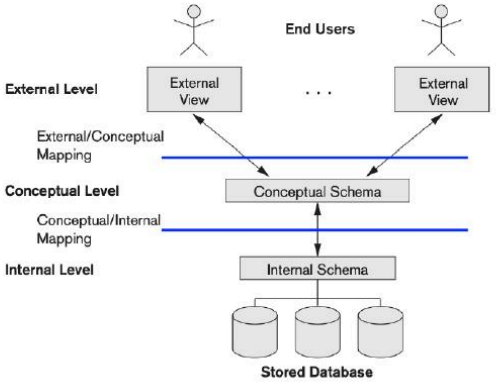
- **Data Model**: DB 구조를 기술하는 개념 모음
  - Conceptual — 사용자가 데이터를 보는 방식에 가까움
  - Representational — 개념과 물리 사이
  - Physical — 컴퓨터에 어떻게 저장되는지
  - (추상화 = 핵심 드러내기 + 저장 세부 감추기)

필기 ✍️

— 넘어가도 되는 쪽

Summary: Database System Architecture

- ❑ **Three-Schema Architecture**
  - ■ Internal schema
    - ❑ Describing the physical storage structure of the database
  - ■ Conceptual schema
    - ❑ Describing the logical structure of the databases
  - ■ External Schema
    - ❑ Describing the part of the database structure
      - ■ Only a particular user group is interested in
      - ■ Hiding the rest of the database from that user group
  - ■ Mapping
    - ❑ Transforming requests and results between Levels
- ❑ **Data Independence**
  - ■ Logical data independence
  - ■ Physical data independence



38

요약: DB 시스템 구조 (계속)

- **3단계 스키마**
  - Internal — 물리 저장 구조
  - Conceptual — 논리 구조
  - External — 특정 사용자 그룹이 관심 있는 부분 (나머지는 감춤)
  - Mapping — 단계 사이 요청·결과 변환
- **데이터 독립성** — Logical / Physical

필기 ✍️

★ 시험 여기까지가 '구조' 파트 — 표로 정리해 두면 서술형 대비 끝

Outline

❑ **Introduction to DB Systems**
- ■ Characteristics and advantages

❑ **DB system Architecture**
- ■ Conceptual, representational, and physical data models
- ■ Three-schema architecture and data independence

❑ **Entity-Relationship (ER) Model**
- ■ Entity types and attributes
- ■ Initial conceptual design
- ■ Relationship types, constraints, and attributes
- ■ Refining conceptual design



39

목차 (Outline)

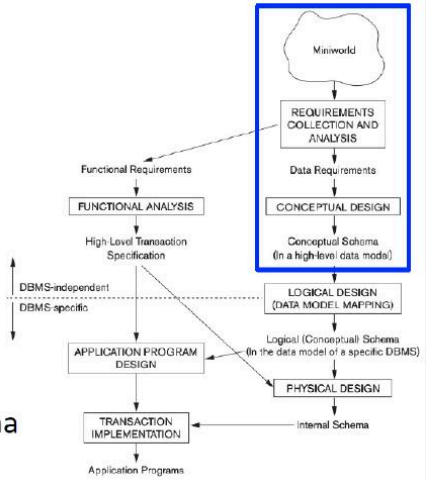
- 이제 세 번째 덩어리: **ER 모델**
  - 엔티티 타입과 속성 / 초기 개념 설계 / 관계 타입·제약·속성 / 개념 설계 다듬기

필기 ✍️

— 넘어가도 되는 쪽

Database Design Process

- **Phase 1: Requirements Collection and Analysis**
  - Characterize the data needs of the prospective DB users
  - Collect and analyze data requirements for the database
    - **Output**: a set of users' detailed requirements
- **Phase 2: Conceptual Design**
  - Choosing a data model
  - Translating the requirements into a conceptual schema
- **Phase 3: Implementation**
  - Implementing the DB based on the conceptual DB schema
    - Logical design: the database schema
    - Physical design: the physical layout of the DB



40

DB 설계 과정

- **1단계: 요구사항 수집·분석**
  - 예상 사용자의 데이터 요구를 파악, 요구사항을 모으고 분석
    - 결과: 사용자 요구사항 명세
- **2단계: 개념 설계**
  - 데이터 모델 고르기
  - 요구사항을 **개념 스키마**로 옮기기
- **3단계: 구현**
  - 개념 스키마를 바탕으로 DB 구현
    - **논리 설계**: DB 스키마 / **물리 설계**: 물리 배치

필기 ✍️

★ 시험 순서: Requirements → conceptual Design → Implementation(Logical → Physical)

Requirement Analysis

- ❑ **What Information Should be Managed?**
  - ■ We want to keep track of a company's employees, departments, and projects
  - ■ **Department**
    - ❑ The company is organized into '**department**'s
      - ■ Each has a unique name, a unique number, locations, manager, start date when that manager began, …
  - ■ **Project**
    - ❑ A department can control a number of '**project**'s
      - ■ Each has a unique name, a unique number, a single location, controlling department
  - ■ **Employee**
    - ❑ '**Employee**'s are working in the company
      - ■ Each has a name, SSN, address, salary, gender, birth date, projects that the employee works on, …
  - ■ **Dependent**
    - ❑ For insurance purposes, the '**dependent**'s of each employee are needed
      - ■ Each has a first name, gender, birth date, relationships to the employee, …

41

요구사항 분석 (회사 DB)

- 회사의 직원·부서·프로젝트를 관리하고 싶다
- **Department**: 고유한 이름, 고유한 번호, 위치(여러 곳), 관리자, 관리자 시작일 …
- **Project**: 부서 하나가 여러 프로젝트를 관리 — 고유한 이름, 고유한 번호, 위치 하나, 관리 부서
- **Employee**: 이름, SSN, 주소, 급여, 성별, 생일, 참여 프로젝트 …
- **Dependent**: 보험용 부양가족 — 이름, 성별, 생일, 직원과의 관계 …

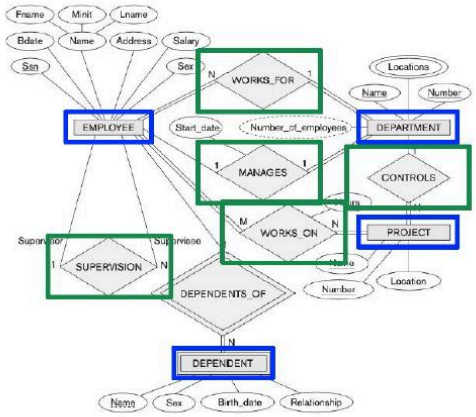
필기 ✍️

💡 이해 요구 문장의 단어가 설계 단서: 'unique' → 키, 'locations'(복수) → 다중값, 'a number of' → N

Conceptual Data Modeling

❑ Entity-Relationship (ER) Model

- Describing data as a collection of ***entities*** and ***relationships***
  - ❑ **Entity**: a “thing” or “object” that is distinguishable from others
  - ❑ **Relationship**: an association among several entities
- Represented diagrammatically by an ***ER diagram***



42

개념적 데이터 모델링 — ER 모델

- **Entity-Relationship (ER) Model**
  - 데이터를 **엔티티**와 **관계**의 모음으로 기술
    - **Entity**: 다른 것과 구별되는 '것'이나 '객체'
    - **Relationship**: 여러 엔티티 사이의 연관
  - **ER diagram**으로 그림으로 나타냄
- 그림: 파란 상자 = 엔티티, 초록 마름모 = 관계

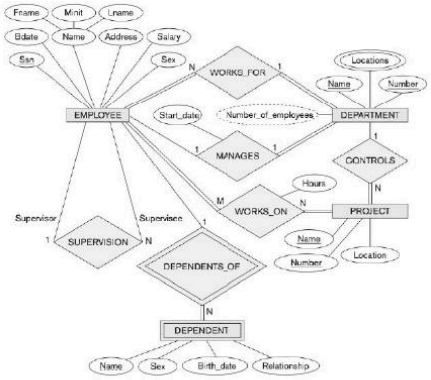
필기 ✍️

★ 시험 ER 다이어그램 기호: 사각형 = 엔티티, 마름모 = 관계, 타원 = 속성

Conceptual Data Modeling (cont.)

❑ **How to Choose a Good Design?**

- **Redundancy**: a bad design may result in repeating of information
- **Incompleteness**: a bad design may make certain aspects of data difficult to model



How can we evaluate DB designs?
What designs are bad or good?



→ Functional dependency and normalization in Week 6

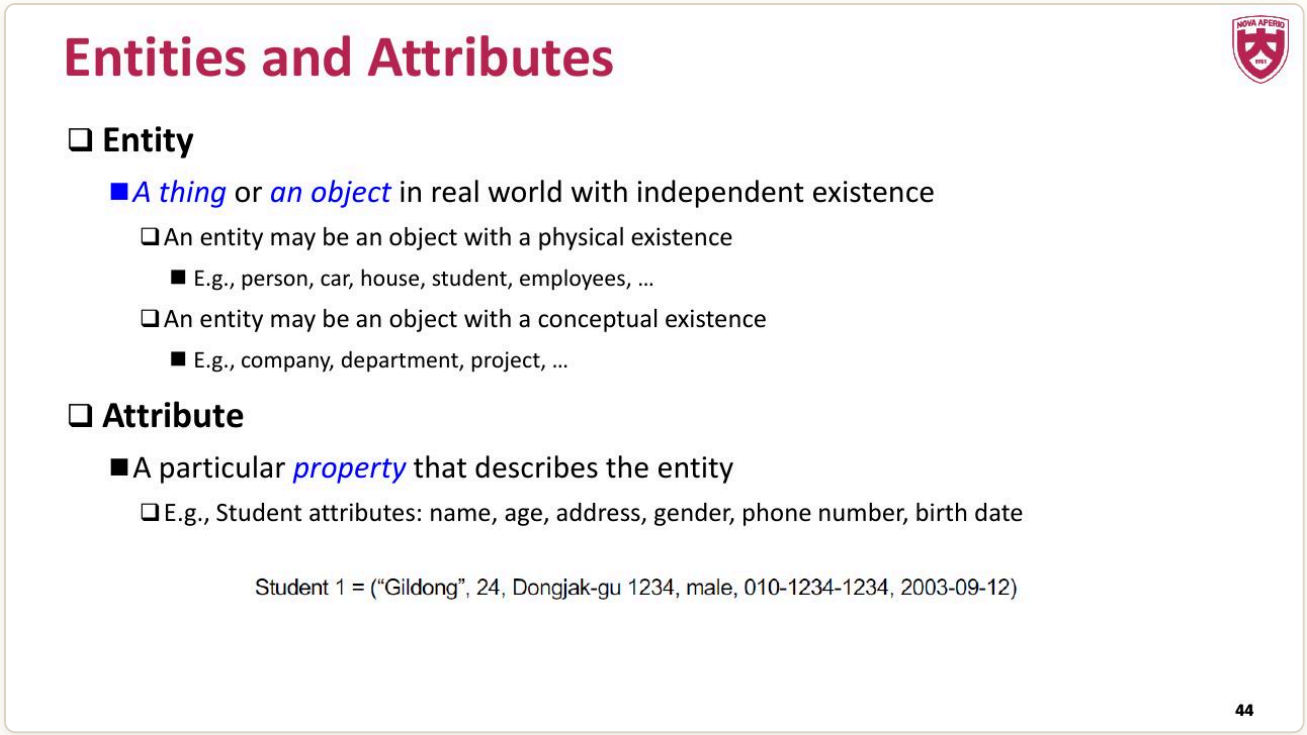
43

개념적 데이터 모델링 (계속)

- **좋은 설계는 어떻게 고르나?**
  - **Redundancy**: 나쁜 설계는 정보가 반복될 수 있다
  - **Incompleteness**: 나쁜 설계는 어떤 데이터를 표현하기 어렵게 만든다
- 그림: DB 설계를 어떻게 평가하지? → 6주차 함수 종속과 정규화

필기 ✍️

★ 시험 나쁜 설계 두 가지: Redundancy · Incompleteness



엔티티와 속성

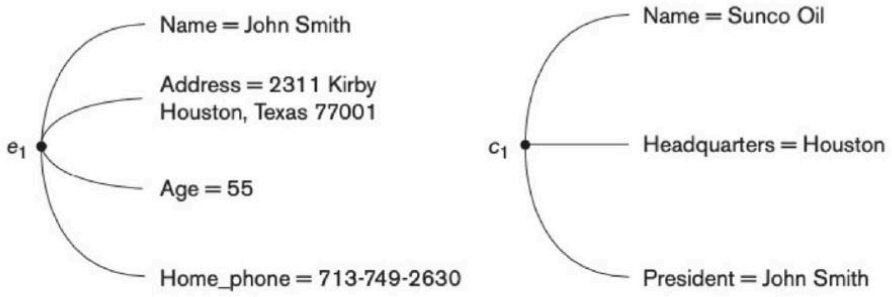
- **Entity**: 현실 세계에서 **독립적으로 존재**하는 것·객체
  - 물리적으로 존재 — 사람, 차, 집, 학생, 직원 …
  - 개념적으로 존재 — 회사, 부서, 프로젝트 …
- **Attribute**: 엔티티를 설명하는 특정 **성질(property)**
  - 예: 학생 속성 — 이름, 나이, 주소, 성별, 전화번호, 생일
  - Student 1 = ("Gildong", 24, Dongjak-gu 1234, male, 010-..., 2003-09-12)

필기 ✍️

⚠ 주의 부서·프로젝트처럼 만질 수 없는 것도 엔티티 (conceptual existence)

Entities and Attributes: Examples

- **Employee**
  - Name, address, age, and home_phone
- **Company**
  - Name, headquarters, and president



45

엔티티와 속성: 예

- Employee — Name, Address, Age, Home_phone
  - e1: John Smith / 2311 Kirby, Houston / 55 / 713-749-2630
- Company — Name, Headquarters, President
  - c1: Sunco Oil / Houston / John Smith

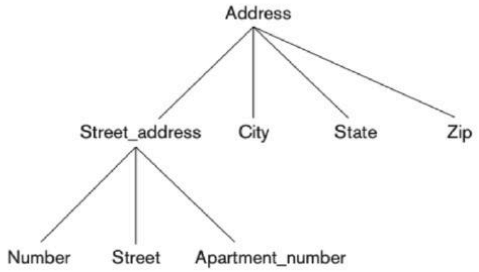
필기 ✍️

— 넘어가도 되는 쪽

Categorization of Attributes

❑ **Category 1: Composite vs Simple Attributes**

- ■ **Composite** attributes: *can be divided* into smaller subparts (*forming a hierarchy*)
  - ❑ E.g., the address attribute that consists of Street_address, city, state, and zip
- ■ **Simple** (atomic) attributes: *non-divisible*
  - ❑ E.g., city, state, zip, number, street, apartment_number



Example of a hierarchy of composite attributes

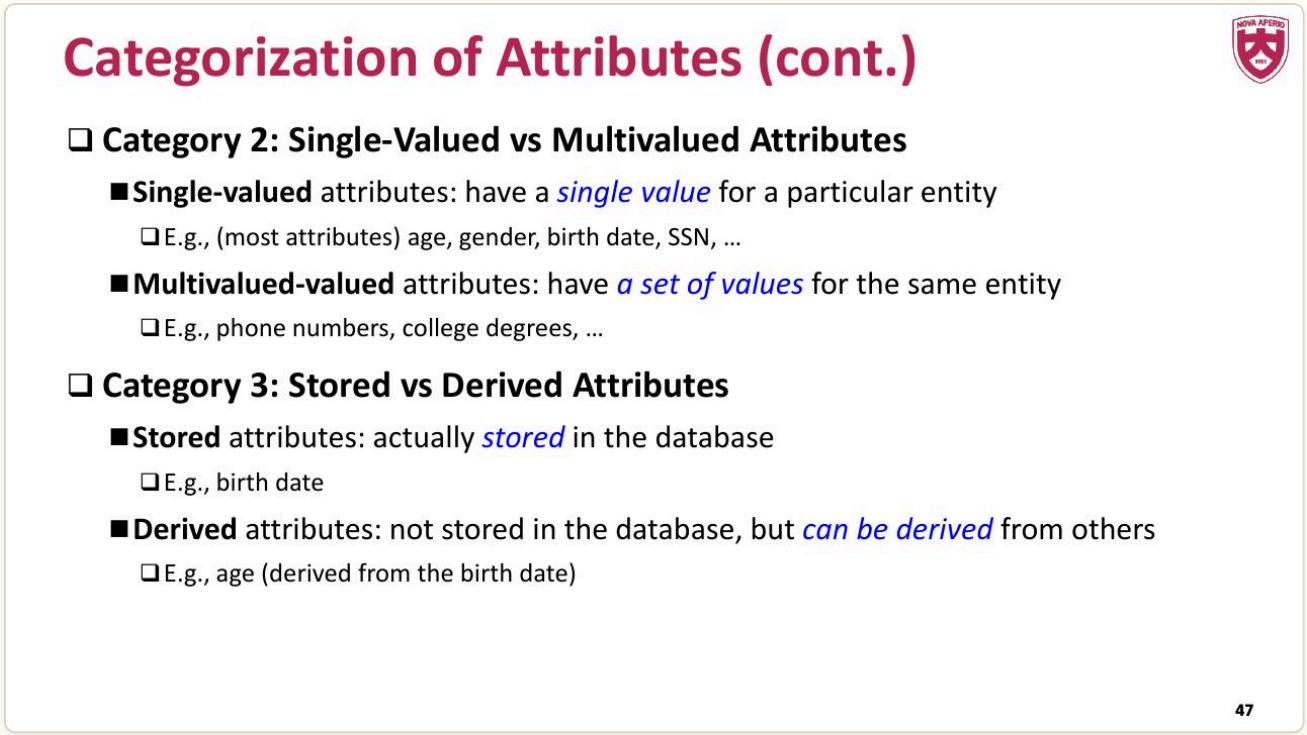
46

속성의 분류 ①

- **Composite vs Simple**
  - **Composite (복합)**: 더 작은 부분으로 나눌 수 있음 (계층을 이룸)
    - 예: Address = Street_address + City + State + Zip
  - **Simple (atomic, 단순)**: 나눌 수 없음
    - 예: City, State, Zip, Number, Street, Apartment_number
- 그림: Address → Street_address(Number, Street, Apartment_number), City, State, Zip

필기 ✍️

★ 시험 복합 속성 예 = 주소·이름(Fname, Minit, Lname)



속성의 분류 ②③

- **Single-valued vs Multivalued**
  - **Single-valued**: 한 엔티티에 값 하나 — 나이, 성별, 생일, SSN (대부분)
  - **Multivalued**: 한 엔티티에 **값 여러 개** — 전화번호들, 학위들
- **Stored vs Derived**
  - **Stored**: DB에 실제 저장 — 생일
  - **Derived**: 저장 안 하고 다른 값에서 **계산** — 나이(생일에서)

필기 ✍️

★ 시험 속성 분류 3쌍: composite/Simple · Single/Multivalued · Stored/Derived

✏ 예·풀이 Age는 Derived, Birth_date는 Stored — 짝으로 나옴

💡 이해 슬라이드 'Multivalued-valued'는 오타 (Multivalued)

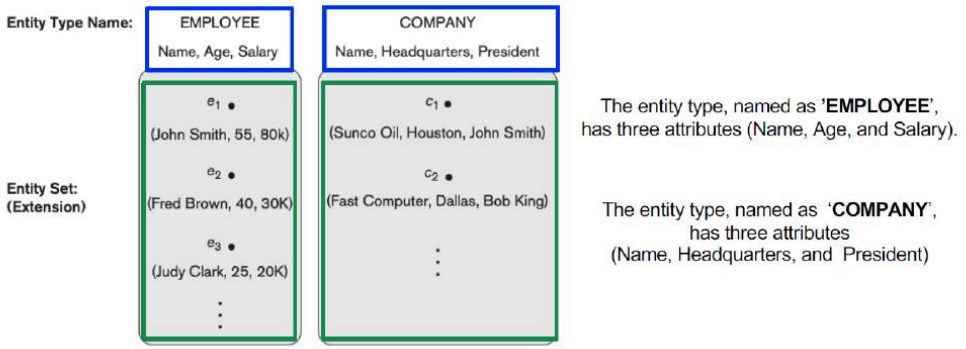
Entity Type and Set

❑ **Entity Type**

■ Described by its name and attributes

❑ **Entity Set**

■ A set of entities that have *the same set of attributes*



48

엔티티 타입과 엔티티 집합

- **Entity Type**: 이름과 속성으로 기술 (틀)
- **Entity Set**: **같은 속성들**을 가진 엔티티들의 집합 (실제 모음, extension)
- 그림: EMPLOYEE(Name, Age, Salary) — e1(John Smith, 55, 80k), e2 …
  - COMPANY(Name, Headquarters, President) — c1(Sunco Oil, Houston, John Smith) …

필기 ✍️

💡 이해 타입 = 붕어빵 틀, 집합 = 구운 붕어빵들

🐟

Key Attributes of an Entity Type

❑ What is a Key Attribute?

- ■ An attribute whose value is ***distinct*** for each individual entity
  - ❑ *I.e., used to identify each entity uniquely*
  - ❑ E.g., student number of STUDENT entity and SSN of PERSON entity
- ■ Multiple attributes can form a key
  - ❑ The combination of the attribute values must be distinct for each entity
- ■ An entity type can have more than two keys

Registration (Number, State), Vehicle_id, Make, Model, Year, {Color}

CAR_1
((ABC 123, TEXAS), TK629, Ford Mustang, convertible, 2004 {red, black})

CAR_2
((ABC 123, NEW YORK), WP9872, Nissan Maxima, 4-door, 2005, {blue})

CAR_3
((VSY 720, TEXAS), TD729, Chrysler LeBaron, 4-door, 2002, {white, blue})

⋮

The "Registration" attribute consists of two attributes

The CAR entity has two keys: Registration and Vehicle_id

49

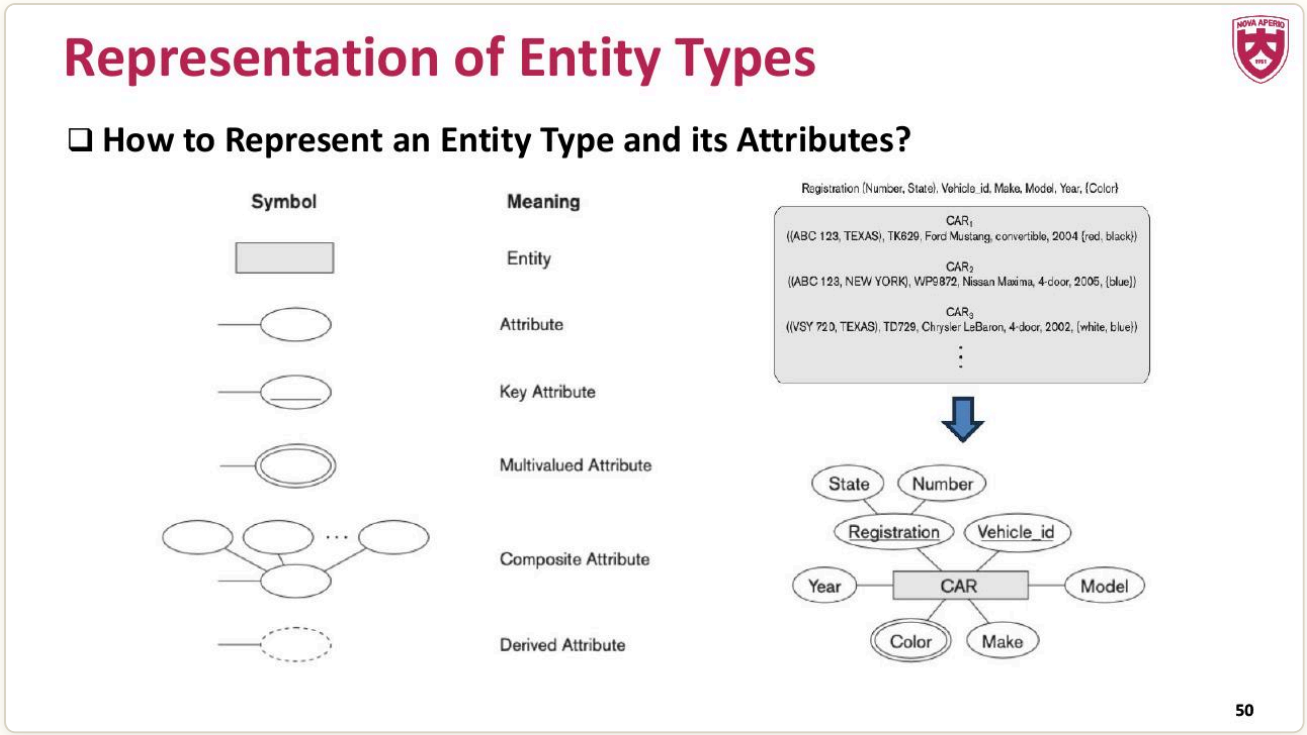
엔티티 타입의 키 속성

- **Key Attribute**: 엔티티마다 값이 **서로 다른** 속성
  - 즉, 각 엔티티를 **유일하게 식별**
  - 예: STUDENT의 학번, PERSON의 SSN
  - **여러 속성이 합쳐 키**가 될 수 있다 — 값의 조합이 엔티티마다 달라야 함
  - 한 엔티티 타입이 **키를 여러 개** 가질 수 있다
- 그림: CAR — Registration(Number, State)과 Vehicle_id 두 개가 키

필기 ✍️

★ 시험 CAR의 키 2개: Registration(복합 키: Number+State), Vehicle_id

⚠ 주의 'ABC 123'은 TEXAS·NEW YORK에 둘 다 있음 → Number 혼자는 키가 아님, State와 합쳐야 키



엔티티 타입 그리는 법

- 기호
  - 사각형 = **Entity**
  - 타원 = **Attribute**
  - 밑줄 타원 = **Key Attribute**
  - **이중** 타원 = **Multivalued** Attribute
  - 타원에 타원이 달림 = **Composite** Attribute
  - **점선** 타원 = **Derived** Attribute
- 그림: CAR — Registration(State, Number) 밑줄, Vehicle_id 밑줄, Color 이중 타원

필기 ✍️

★ 시험 기호 표 통째로 외우기: 밑줄=키, 이중=다중값, 점선=유도

Initial Conceptual Design: COMPANY

- ❑ **DEPARTMENT**
  - ■ **Attributes**: Name, Number, Locations, Manager, and Manager_start_date
    - ❑ Name and Number are key attributes (underlined) since each was specified to be unique
    - ❑ Locations is the multivalued attributed
- ❑ **PROJECT**
  - ■ **Attributes**: Name, Number, Location, Controlling_department
    - ❑ NAME and Number are key attributes



51

초기 개념 설계: COMPANY ①

- **DEPARTMENT** 속성: Name, Number, Locations, Manager, Manager_start_date
  - Name과 Number는 **키 속성**(밑줄) — 각각 유일하다고 했으므로
  - Locations는 **다중값 속성**
- **PROJECT** 속성: Name, Number, Location, Controlling_department
  - Name과 Number가 키 속성

필기 ✍️

✏ 예.풀이 요구사항의 'unique'가 그대로 밑줄(키)이 됨

Initial Conceptual Design: COMPANY (cont.)

❑ **EMPLOYEE**

■ **Attributes**: Name, SSN, Address, Salary, Birth_date, Sex, Department, Supervisor, Works_on

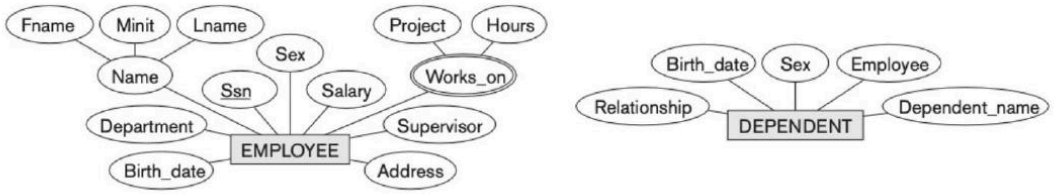
❑ SSN is a key attribute , Name is a composite attribute (Fname, Minit, Lname)

❑ An employee can work on several projects,

■ The number of hours per week an employee works on each project is managed

❑ **DEPENDENT**

■ **Attributes**: Employee, Depedent_name, Sex, Birth_date, Relationship



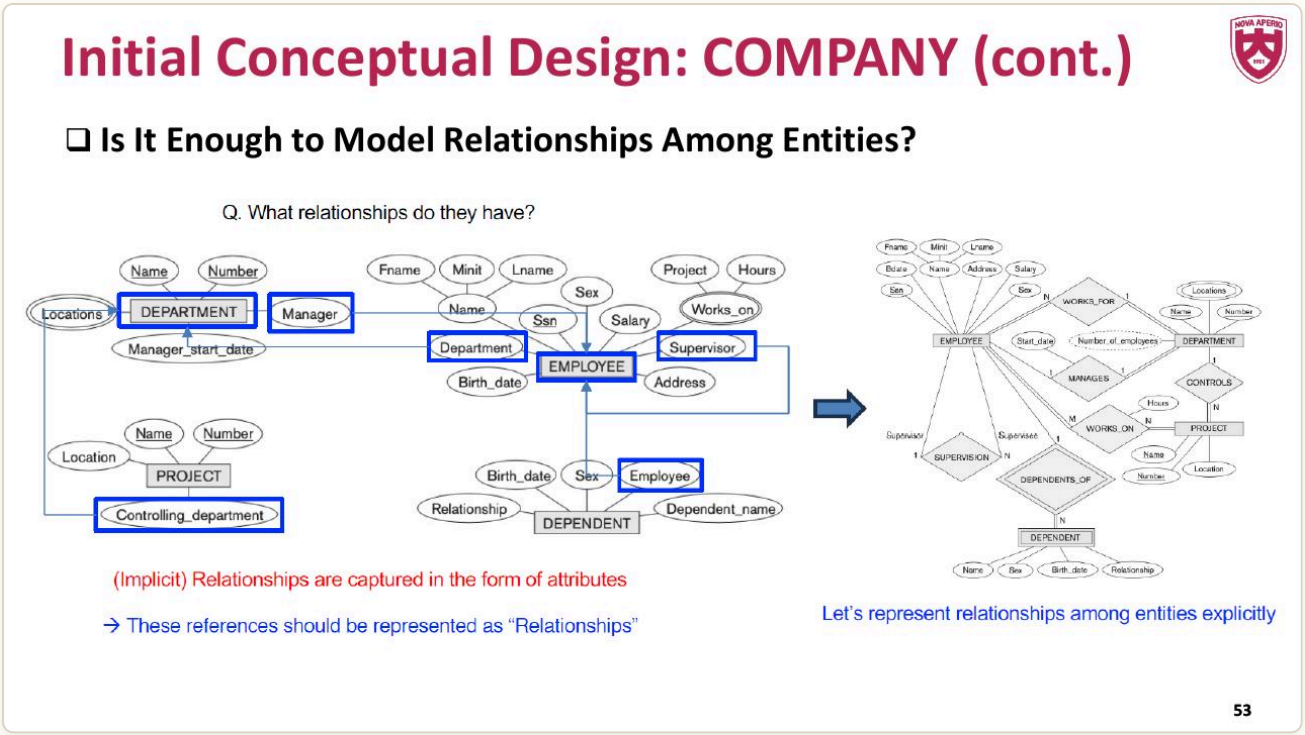
52

초기 개념 설계: COMPANY ②

- **EMPLOYEE** 속성: Name, SSN, Address, Salary, Birth_date, Sex, Department, Supervisor, Works_on
  - SSN은 키, Name은 **복합 속성**(Fname, Minit, Lname)
  - 직원은 여러 프로젝트에서 일할 수 있고, 프로젝트별 **주당 근무 시간**도 관리
- **DEPENDENT** 속성: Employee, Dependent_name, Sex, Birth_date, Relationship

필기 ✍️

💡 이해 works_on(Project, Hours)가 다중값 +복합으로 묶여 있음 ⟶ 뒤에서 관계로 바꿈



초기 개념 설계: COMPANY ③

- **엔티티 사이 관계를 이렇게 표현해도 충분할까?**
- 그림: Manager, Department, Supervisor, Employee, Controlling_department 같은 속성이 사실은 다른 엔티티를 가리킴
  - (암시적) 관계가 **속성 모양으로** 숨어 있다
  - → 이런 참조는 '**관계(Relationship)**'로 나타내야 한다
  - 엔티티 사이 관계를 **명시적으로** 그리자

필기 ✍️

★ 시험 핵심 아이디어: 다른 엔티티를 가리키는 속성
⟶ 관계 타입으로 바꾼다 (p.72~79에서 실행)

Relationship Types, Sets, and Instances

❑ Relationship Type

- ■ Described by its name and participating entity types
- ■ Define a set of associations (i.e., ***relationship set***) among *participating entity types*

❑ Relationship Set

- ■ A set of ***relationship instances*** r_i
  - ❑ r_i associates *n* individual entities (e_1, e_2, …, e_n), and each e_j in r_i is a member of entity set E_j

$$r_i = (e_1, e_2, \ldots, e_n)$$
$$e_1 \in E_1, e_2 \in E_2, \ldots, e_n \in E_n$$

  - ❑ The entity sets $E_1, E_2, \ldots, E_n$ are said to **participate** in the relationship type R

e.g., "**Works_for**" Relationship $r_i = (e_1, e_2)$,
→ Employee e_1 "works for" Department e_2

54

관계 타입·집합·인스턴스

- **Relationship Type**
  - 이름과 **참여하는 엔티티 타입들**로 기술
  - 참여 엔티티 타입 사이 연관들의 집합(= relationship set)을 정의
- **Relationship Set**
  - **relationship instance r_i** 들의 집합
    - $r_i = (e_1, e_2, \ldots, e_n)$ — 각 e_j는 엔티티 집합 E_j의 원소
    - 엔티티 집합 $E_1 \ldots E_n$이 관계 타입 R에 **참여(participate)**한다고 함
  - 예: Works_for 관계 $r_i = (e_1, e_2)$ → 직원 e_1이 부서 e_2에서 일함

필기 ✍️

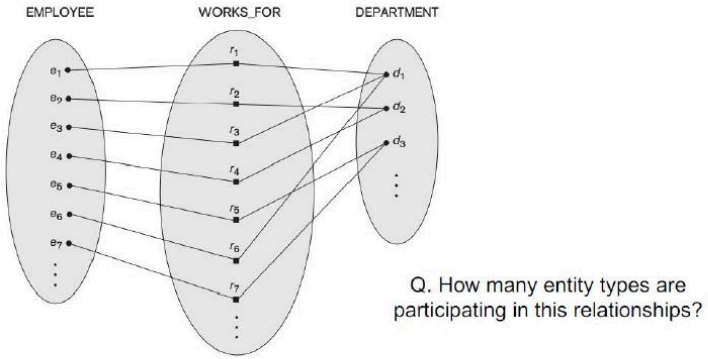
💡 이해 관계 타입 = 동사 '일한다', 관계 인스턴스 = '김철수가 영업부에서 일한다' 한 줄

Relationship Type (Example)

❑ **Relationship Type: WORKS_FOR**

■ Relationships between EMPLOYEE and DEPARTMENT

❑ Associating each employee with the department for which the employee works



Q. How many entity types are participating in this relationships?

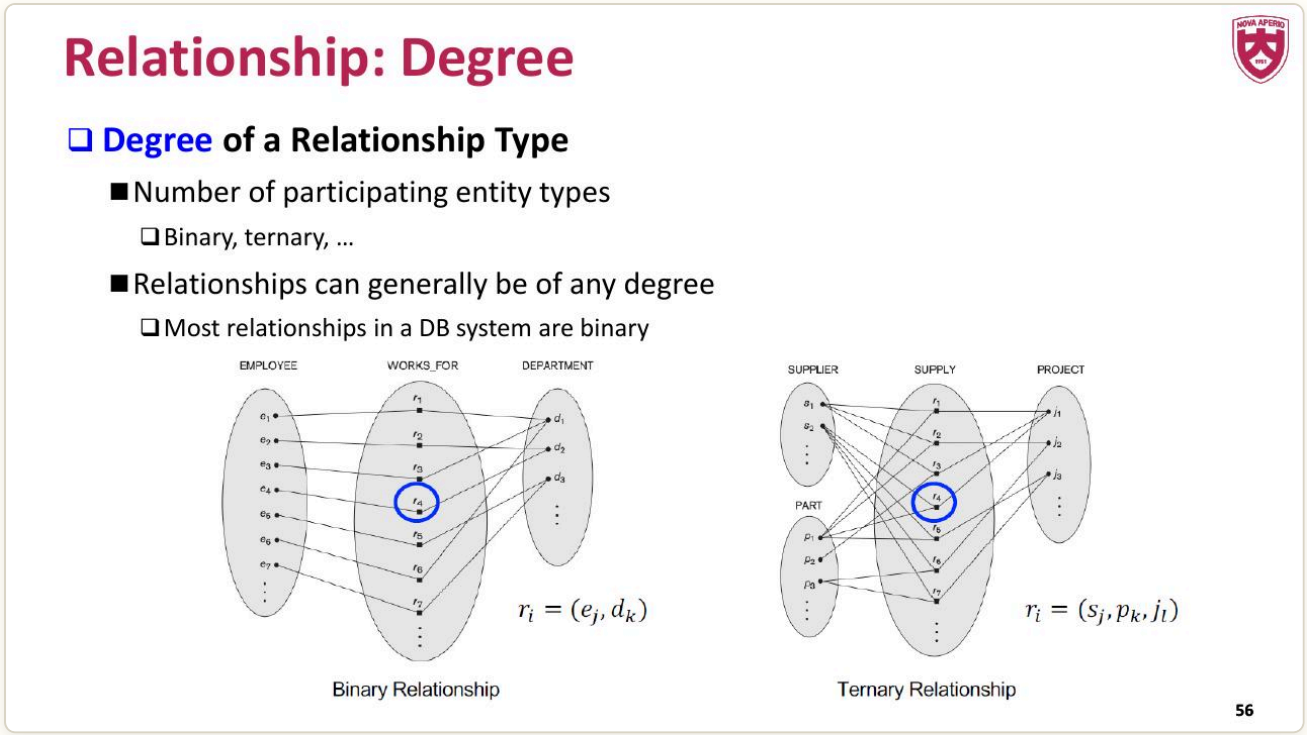
55

관계 타입 예: WORKS_FOR

- EMPLOYEE와 DEPARTMENT 사이 관계
  - 각 직원을 그 직원이 일하는 부서와 연결
- 그림 Q. 이 관계에 참여하는 엔티티 타입은 몇 개?

필기 ✍️

✏ 예.풀이 답: 2개 (EMPLOYEE, DEPARTMENT) → 이항(binary)



관계의 차수 (Degree)

- **Degree**: 참여하는 **엔티티 타입의 수**
  - Binary(이항), Ternary(삼항), ...
  - 관계는 몇 차수든 될 수 있지만
    - **DB의 관계는 대부분 이항(binary)**
- 그림: WORKS_FOR(이항) $r_i = (e_j, d_k)$ / SUPPLY(삼항: SUPPLIER, PART, PROJECT) $r_i = (s_j, p_k, j_1)$

필기 ✍️

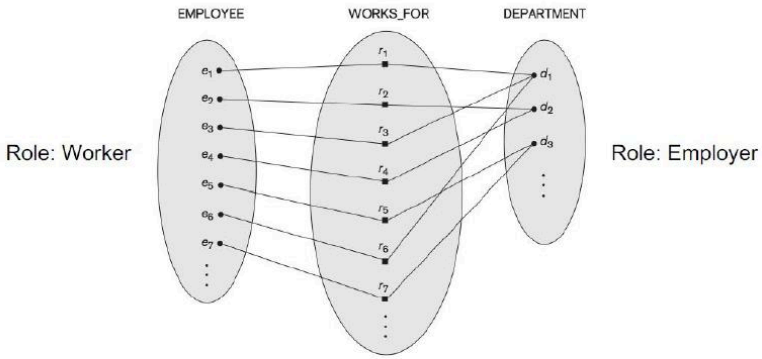
⚠ 주의 차수 = 엔티티 '타입' 수. 인스턴스(선) 개수가 아님!

Relationship: Role Names

❑ **Role Name of a Relationship Type**

■ Signifying a role that a participating entity plays in each relationship instance

❑ E.g., WORKS_FOR relationships between EMPLOYEE and DEPARTMENT



→ It helps us to understand what the relationship means

57

관계의 역할 이름 (Role Names)

- **Role name**: 참여 엔티티가 각 관계 인스턴스에서 맡는 **역할**
  - 예: WORKS_FOR에서 EMPLOYEE = Worker, DEPARTMENT = Employer
  - → 관계의 뜻을 이해하는 데 도움

필기 ✍️

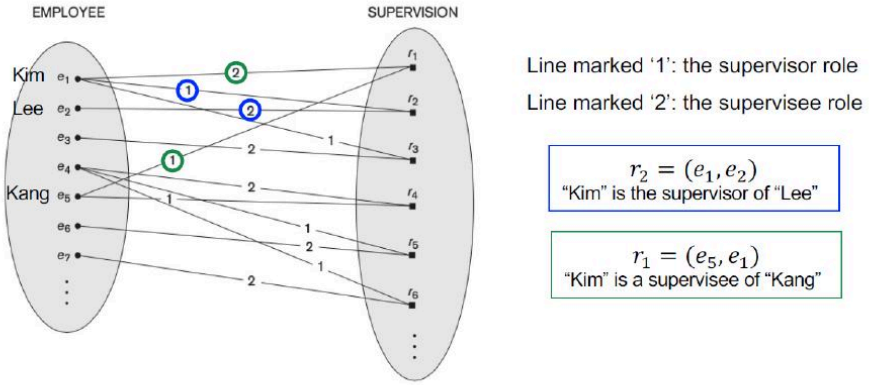
— 넘어가도 되는 쪽

Relationship: Recursive Relationships

❑ Recursive Relationship

■ The same entity type participates more than once in a relationship type

❑ Essential to specify the role name for distinguishing the participating entities



58

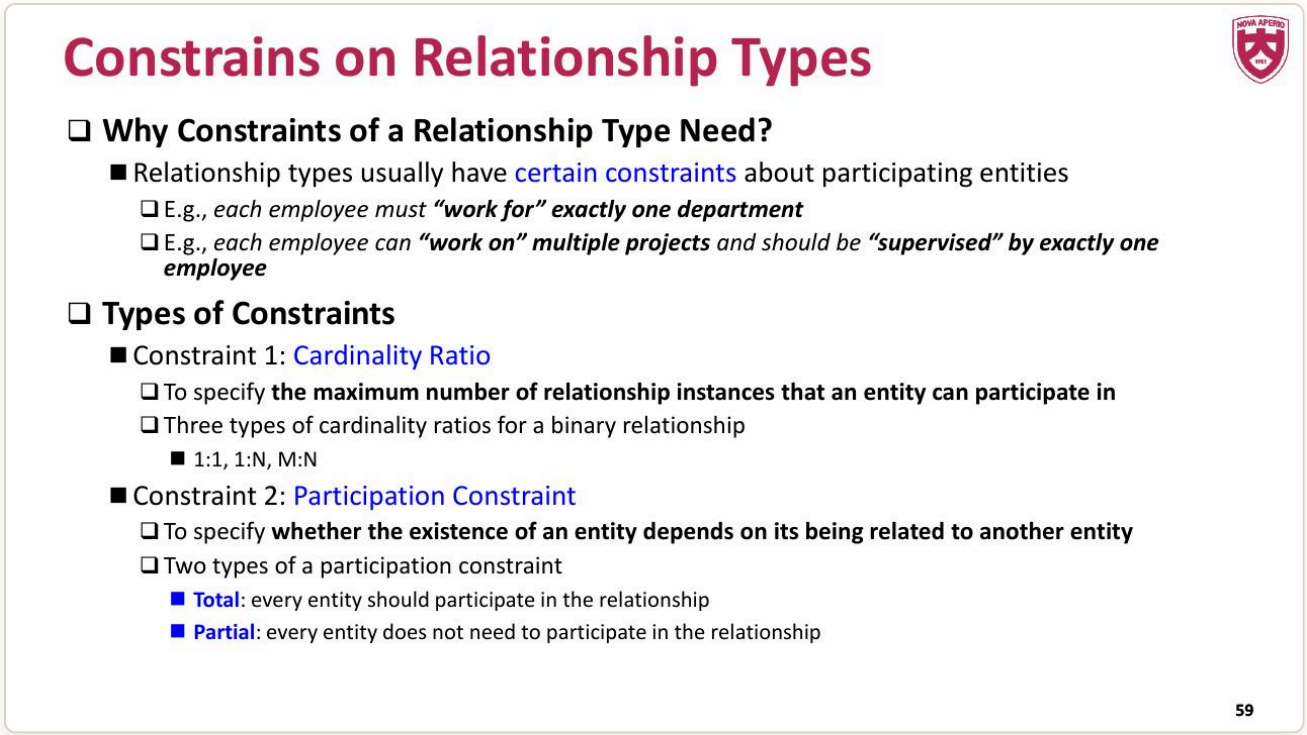
순환 관계 (Recursive)

- **Recursive relationship**: **같은 엔티티 타입이 한 관계에 두 번 이상** 참여
  - 참여 엔티티를 구분하려면 **역할 이름이 꼭 필요**
- 그림: SUPERVISION — 선 '1' = supervisor(상사) 역할, 선 '2' = supervisee(부하) 역할
  - $r_2 = (e_1, e_2)$: Kim이 Lee의 상사
  - $r_1 = (e_5, e_1)$: Kim은 Kang의 부하

필기 ✍️

★ 시험 순환 관계에선 role name이 essential

✏ 예.풀이 같은 Kim이 r_2에선 상사, r_1에선 부하
— 역할이 없으면 누가 누군지 모름



Constrains on Relationship Types

❑ **Why Constraints of a Relationship Type Need?**

- Relationship types usually have certain constraints about participating entities
  - E.g., *each employee must* ***"work for" exactly one department***
  - E.g., *each employee can* ***"work on" multiple projects*** *and should be* ***"supervised" by exactly one employee***

❑ **Types of Constraints**

- Constraint 1: Cardinality Ratio
  - To specify **the maximum number of relationship instances that an entity can participate in**
  - Three types of cardinality ratios for a binary relationship
    - 1:1, 1:N, M:N
- Constraint 2: Participation Constraint
  - To specify **whether the existence of an entity depends on its being related to another entity**
  - Two types of a participation constraint
    - **Total**: every entity should participate in the relationship
    - **Partial**: every entity does not need to participate in the relationship

59

관계 타입의 제약

- 관계 타입에는 보통 참여 엔티티에 대한 **제약**이 있다
  - 예: 직원은 **정확히 한** 부서에서 일해야 한다
  - 예: 직원은 여러 프로젝트에서 일할 수 있고, 상사는 **정확히 한** 명
- **제약 1: Cardinality Ratio (카디널리티 비율)**
  - 엔티티가 참여할 수 있는 관계 인스턴스의 **최대 수**
  - 이항 관계엔 세 가지 — **1:1, 1:N, M:N**
- **제약 2: Participation Constraint (참여 제약)**
  - 엔티티의 존재가 다른 엔티티와의 관계에 **의존하는지**
  - **Total**: 모든 엔티티가 참여해야 함 / **Partial**: 모두 참여할 필요는 없음

필기 ✍️

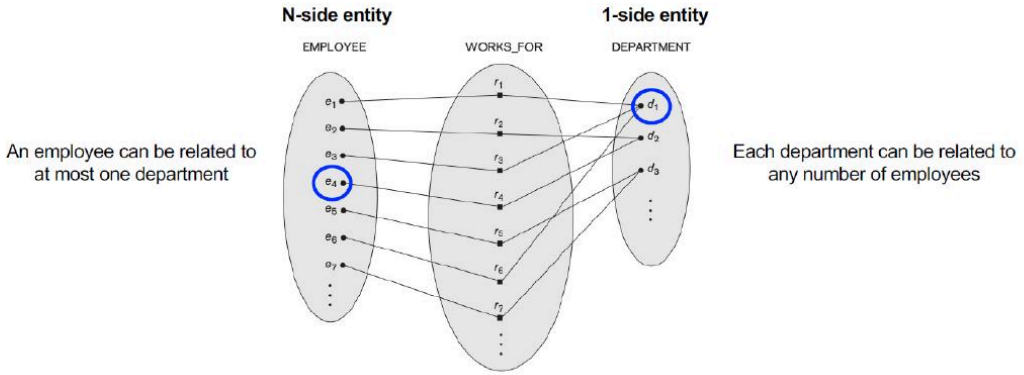
★ 시험 cardinality = 최대(maximum) / Participation = 최소(꼭 참여하나?)

⚠ 주의 두 제약을 바꿔 설명하면 틀림 — 시험 단골 함정

Constraint 1: Cardinality Ratio

❑ 1:N Cardinality Ratio

■A 1:N binary relationship relates an entity to any number of entities



Real-world Constraints:
(1) An employee can work for only one department
(2) A department can employ any number of employees

60

제약 1: 카디널리티 비율 — 1:N

- 1:N 이항 관계는 한 엔티티를 **여러** 엔티티와 연결
- 그림: WORKS_FOR — EMPLOYEE가 N쪽, DEPARTMENT가 1쪽
  - 직원은 **최대 한** 부서와 연결 / 부서는 **몇 명이든** 직원과 연결
- 현실 제약: (1) 직원은 한 부서에서만 일함 (2) 부서는 직원을 몇 명이든 둠

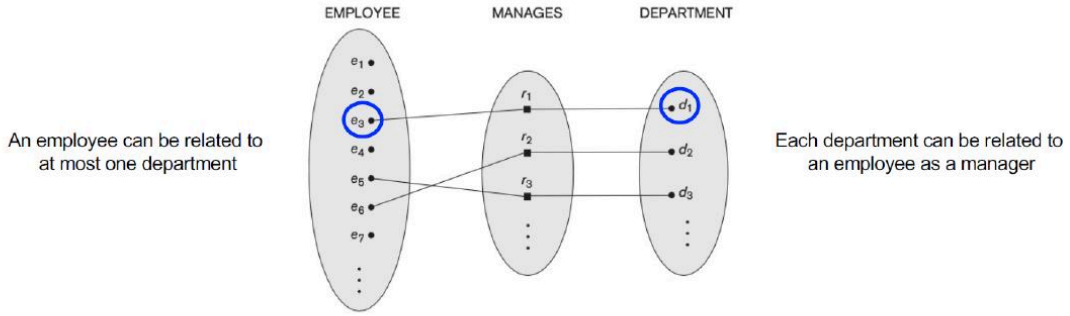
필기 ✍️

💡 이해 'N쪽' = 여럿이 몰리는 쪽(직원), '1쪽' = 하나뿐인 쪽(부서)

Constraint 1: Cardinality Ratio (cont.)

❑ 1:1 Cardinality Ratio

■A 1:1 binary relationship relates an entity to another entity



Real-world Constraints:
(1) An employee can manage at most one department
(2) A department can have at most one manager

61

카디널리티 비율 — 1:1

- 1:1 이항 관계는 한 엔티티를 **다른 엔티티 하나**와 연결
- 그림: MANAGES — 직원은 최대 한 부서를 관리 / 부서는 관리자 한 명
- 현실 제약: (1) 직원은 최대 한 부서를 관리 (2) 부서는 관리자 최대 한 명

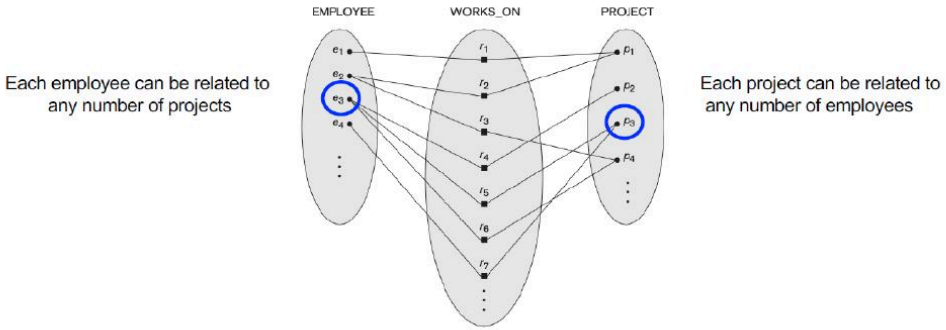
필기 ✍️

— 넘어가도 되는 쪽

Constraint 1: Cardinality Ratio (cont.)

❑ M:N Cardinality Ratio

■A M:N binary relationship relates any number of entities to any number of entities



Real-world Constraints:
(1) An employee can work on any number of projects
(2) A project can be conducted by any number of employees

62

카디널리티 비율 — M:N

- M:N 이항 관계는 **몇 개든** 엔티티를 **몇 개든** 엔티티와 연결
- 그림: WORKS_ON — 직원은 프로젝트 여러 개 / 프로젝트는 직원 여러 명
- 현실 제약: (1) 직원은 프로젝트 몇 개든 (2) 프로젝트는 직원 몇 명이든

필기 ✍️

★ 시험 회사 DB 짝: MANAGES 1:1 · WORKS_FOR 1:N · WORKS_ON M:N

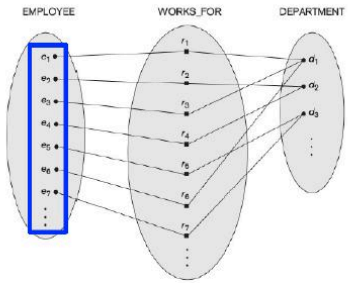
Constraint 2: Participation Constraint

❑ **Total** Participation

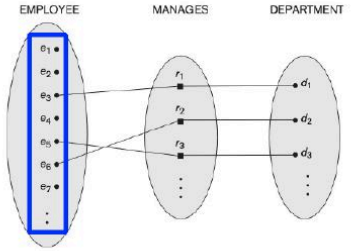
■ Every entity should participate in the relationship

❑ **Partial** Participation

■ Every entity does not need to participate in the relationship



The participation of EMPLOYEE in WORKS_FOR:
Total Participation



The participation of EMPLOYEE in MANAGES:
Partial Participation

63

제약 2: 참여 제약

- **Total Participation (전체 참여)**: 모든 엔티티가 관계에 참여해야 함
- **Partial Participation (부분 참여)**: 모든 엔티티가 참여할 필요는 없음
- 그림: WORKS_FOR에서 EMPLOYEE = **Total** (모든 직원이 어느 부서엔 속함)
  - MANAGES에서 EMPLOYEE = **Partial** (관리자가 아닌 직원도 있음)

필기 ✍️

★ 시험 ER 그림에서 Total = 이중선(═), Partial = 단일선(—)

✏ 예.풀이 같은 EMPLOYEE도 관계마다 참여가 다름: WORKS_FOR total, MANAGES partial

Exercise 1: Cardinality Ratio

❑ Designing the Cardinality Ratio of a Relationship Type

■ For the following Binary Relationships, suggest cardinality ratios based on the common-sense meaning of the entity types

Entity 1	Cardinality Ratio	Entity 2
STUDENT	_____	SOCIAL_SECURITY_CARD
STUDENT	_____	TEACHER
COUNTRY	_____	CURRENT_PRESIDENT
COURSE	_____	TEXTBOOK
ITEM (that can be found in an order)	_____	ORDER
STUDENT	_____	CLASS
CLASS	_____	INSTRUCTOR

64

연습 1: 카디널리티 비율

- 상식적인 뜻에 맞게 이항 관계의 카디널리티 비율을 정하라
  - STUDENT — SOCIAL_SECURITY_CARD
  - STUDENT — TEACHER
  - COUNTRY — CURRENT_PRESIDENT
  - COURSE — TEXTBOOK
  - ITEM(주문에 들어가는) — ORDER
  - STUDENT — CLASS
  - CLASS — INSTRUCTOR

필기 ✍️

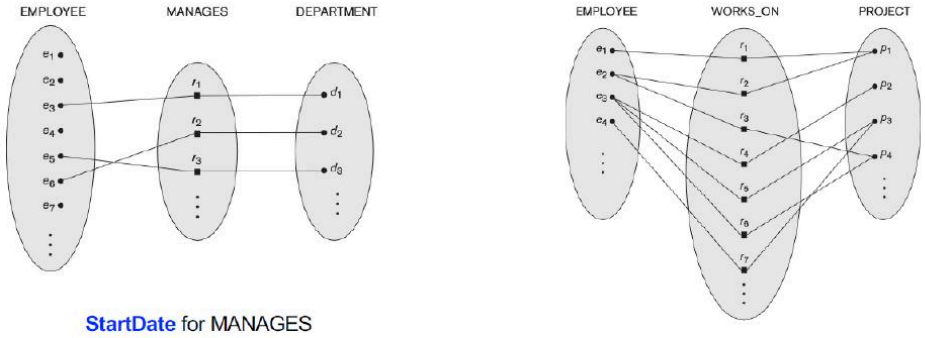
✏ 예.풀이 풀이 예시(교안에 정답 없음, 상식 기준): 학생-SSN카드 1:1 · 학생-교사 M:N · 국가-현직 대통령 1:1 · 과목-교재 M:N · 품목-주문 M:N · 학생-수업 M:N · 수업-강사 N:1

⚠ 주의 정답은 '가정'에 따라 달라질 수 있음 — 답과 함께 가정을 한 줄 쓰기

Attributes of Relationship Types

❑ Relationship Types Can Have Attributes

The company wants to keep track of:
(1) the **start date** of a manager when the employee manage the department
(2) the **number of working hours** of each employee on a specific project and



StartDate for MANAGES

Hours for WORKS_ON

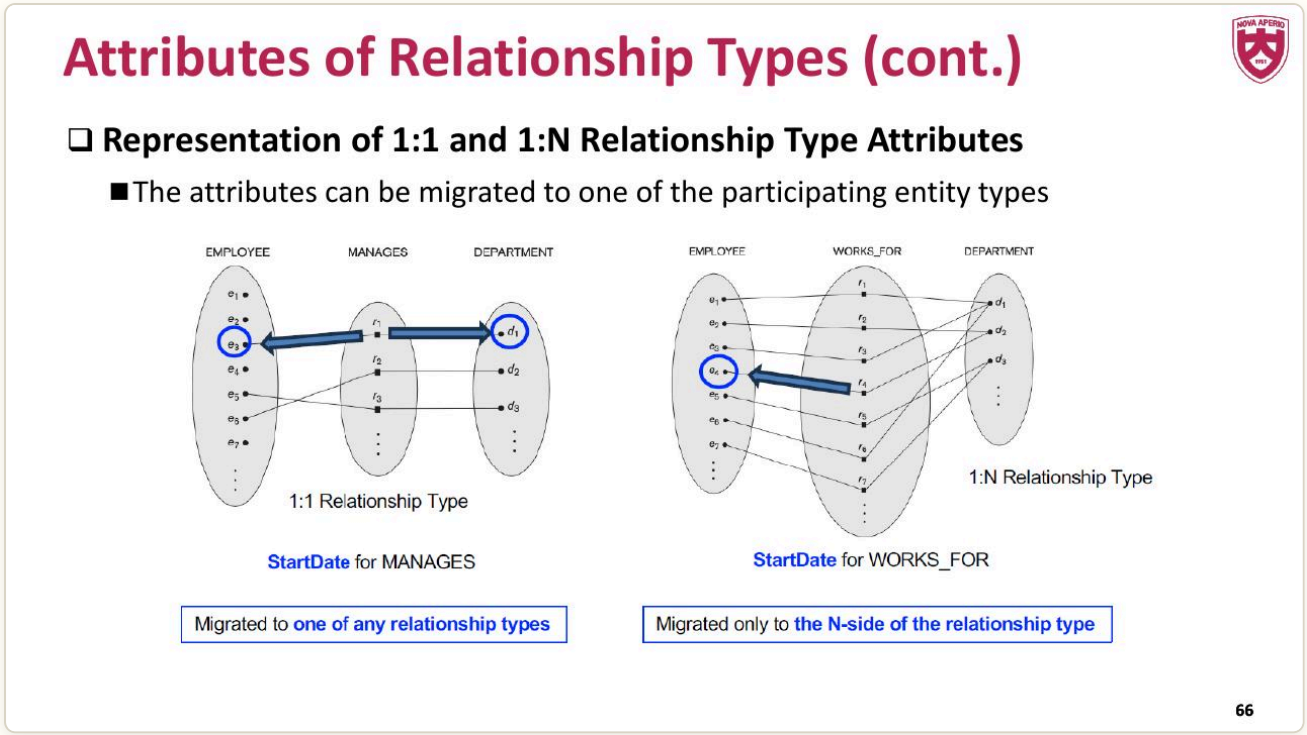
65

관계 타입의 속성

- **관계 타입도 속성을 가질 수 있다**
- 회사가 기록하고 싶은 것:
  - (1) 직원이 부서를 관리하기 시작한 **날짜** → MANAGES의 StartDate
  - (2) 직원이 특정 프로젝트에서 일하는 **시간** → WORKS_ON의 Hours

필기 ✍️

★ 시험 관계 속성 대표 예 2개:
StartDate(MANAGES), Hours(WORKS_ON)



관계 타입의 속성 (계속) — 1:1, 1:N

- 1:1과 1:N 관계의 속성은 **참여 엔티티 쪽으로 옮길 수** 있다
- 그림: 1:1 MANAGES의 StartDate → **어느 쪽이든** 옮길 수 있음
  - 1:N WORKS_FOR의 StartDate → **N쪽(EMPLOYEE)으로만** 옮길 수 있음

필기 ✍️

★ 시험 1:1 ⟶ 아무 쪽 / 1:N ⟶ N쪽만 / M:N ⟶ 못 옮김(다음 쪽)

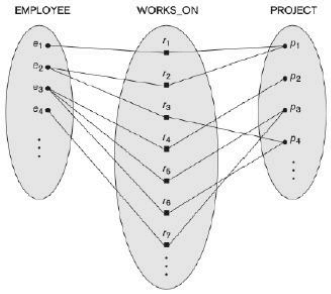
💡 이해 1쪽(부서)에 두면 직원마다 다른 값을 칸 하나에 못 담음 ⟶ N쪽에 둔다

Attributes of Relationship Types (Cont.)

❑ **Representation of M:N Relationship Type Attributes**

■ Some attributes may be determined only by the *combination of participating entities* in a relationship instance, *not by any single entity*

❑ *I.e., Those attributes cannot be migrated to any participating entities*



Hours for WORKS_ON

The number of hours per week an employee 'works on' a project is determined by an employee-project combination.

Hours cannot be migrated into one of the participating entities

67

관계 타입의 속성 (계속) — M:N

- 어떤 속성은 관계 인스턴스의 **참여 엔티티 조합**으로만 정해진다 — **어느 한 엔티티로는 안 됨**
  - 즉, 그런 속성은 참여 엔티티로 **옮길 수 없다**
- 그림: 직원이 프로젝트에서 일하는 주당 시간은 **직원–프로젝트 조합**으로 정해진다
  - **Hours는 참여 엔티티 어디로도 옮길 수 없다**

필기 ✍️

★ 시험 M:N 관계 속성은 관계에 쓴다 (Hours)

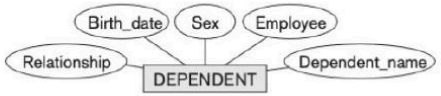
Weak Entity Types

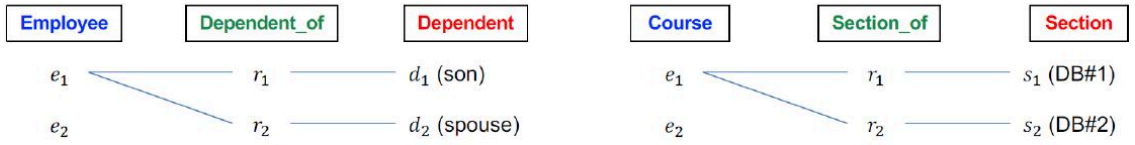
❑ **What is a Weak Entity Type?**

- ■ An entity type has no key attributes of its own inside the entity type
- ■ A weak entity type depends on its *identifying or owner entity type*
  - ❑ Each weak entity must be associated with an identifying entity

❑ **Identifying Relationship**

- ■ Relating a weak entity type to its owner entity type
- ■ A weak entity type always has a *total participation constraint*





68

약한 엔티티 타입 (Weak Entity)

- **Weak entity type**: 자기 안에 **키 속성이 없는** 엔티티 타입
  - **식별(owner) 엔티티 타입**에 의존한다
    - 약한 엔티티마다 식별 엔티티와 꼭 연결돼야 함
- **Identifying Relationship (식별 관계)**
  - 약한 엔티티 타입을 그 owner 엔티티 타입과 잇는 관계
  - 약한 엔티티 타입은 **언제나 전체 참여(total)**
- 그림: Employee–Dependent_of–Dependent(아들, 배우자) / Course–Section_of–Section(DB#1, DB#2)

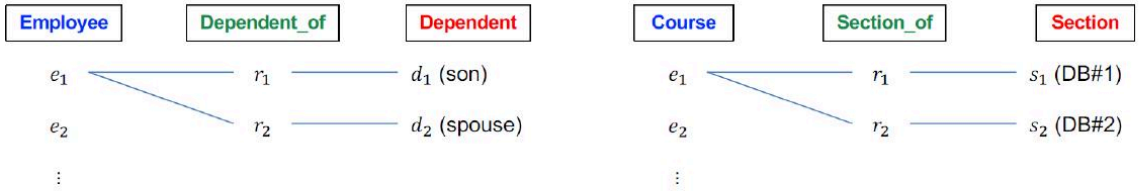
필기 ✍️

★ 시험 약한 엔티티 = 자기 키 없음 + owner에 의존 + 항상 total participation

Weak Entity Types (cont.)

❑ **Partial Key**

- ■ To uniquely identify weak entities that are related to the same owner entity
- ■ Example 1: EMPLOYEE-DEPENDENT
  - ❑ The **Dependent_Name** of DEPENDENT is the partial key since no two dependents have the same name
- ■ Example 2: COURSE-SECTION
  - ❑ The **Section_Num** of Section is the partial key since no two sections have the same number



However, a partial key alone cannot identify its corresponding weak entity
Thus, it can be used as *a key together with thea identifying relationship*

69

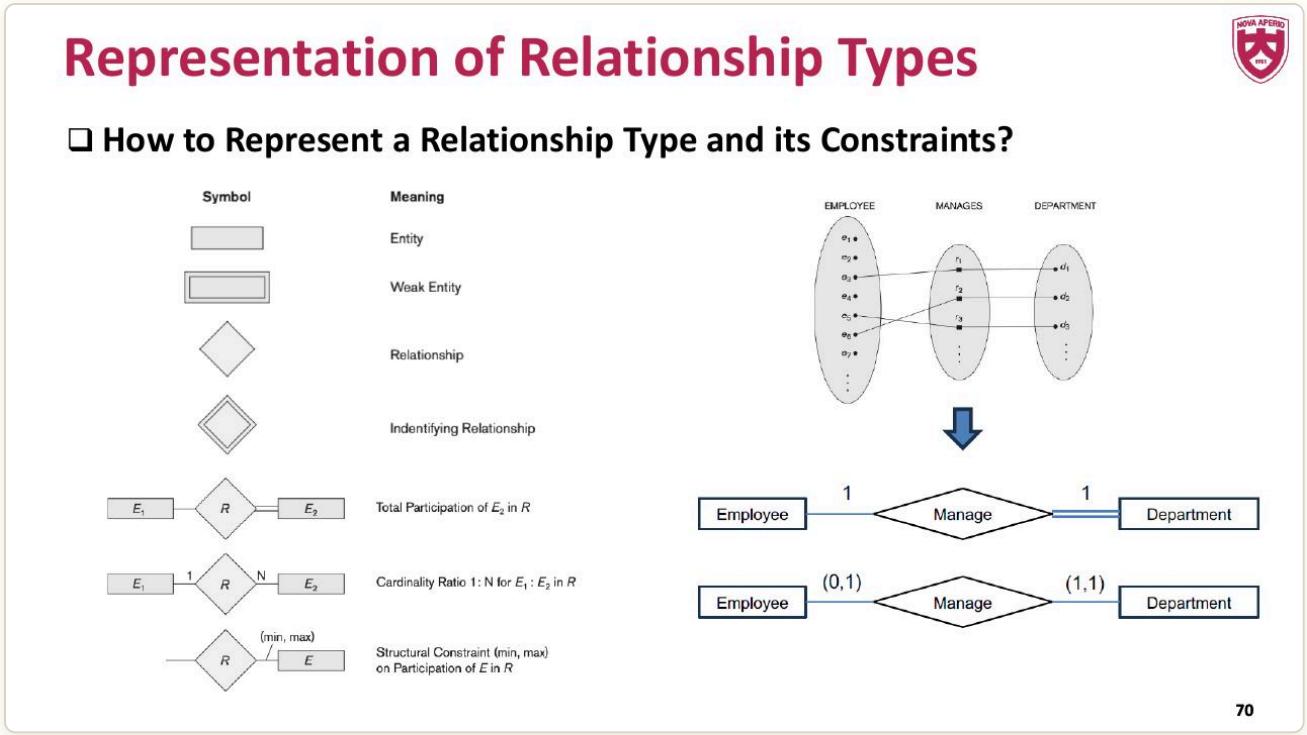
약한 엔티티 타입 (계속) — 부분 키

- **Partial Key (부분 키)**: **같은 owner에 딸린** 약한 엔티티끼리 구별하는 속성
  - 예 1: EMPLOYEE–DEPENDENT — DEPENDENT의 **Dependent_Name** (한 직원의 부양가족끼리는 이름이 안 겹친다고 가정)
  - 예 2: COURSE–SECTION — SECTION의 **Section_Num**
- 하지만 부분 키 **혼자서는** 약한 엔티티를 식별할 수 없다
  - → **식별 관계(owner의 키)와 함께** 키로 쓴다

필기 ✍️

★ 시험 부분 키는 점선 밑줄로 그림. 혼자는 키 아님 — owner 키 + 부분 키 = 식별

✏ 예.풀이 'DB#1 분반'은 과목마다 있음 → 과목번호 + 분반번호여야 한 분반



관계 타입 그리는 법

- 기호
  - 사각형 = Entity / **이중 사각형 = Weak Entity**
  - 마름모 = Relationship / **이중 마름모 = Identifying Relationship**
  - E_2 쪽 **이중선** = R에서 E_2의 **전체 참여**
  - 선 위 1, N = E_1:E_2 카디널리티 비율 1:N
  - **(min, max)** = R에서 E의 참여에 대한 **구조적 제약**
- 그림: Employee —1— Manage —1= Department ⇔ Employee (0,1) Manage (1,1) Department

필기 ✍️

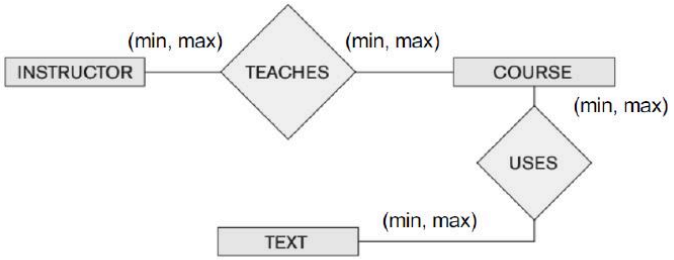
★ 시험 (min,max): min 0 = partial, min ≥1 = total / max = 카디널리티

⚠ 주의 (min,max)는 '자기 엔티티 덮'에 적는다 — 1:N 표기와 위치가 반대라 헷갈림

Exercise 2: Constraints on ER Diagram

❑ **Suggest the (min, max) constraints on the following diagram**

- A course may or may not use a textbook
- A course may not use more than five books
- A course can be taught by at least one instructor and by a maximum of 4 instructors
- A textbook is used in some courses
- An instructor teaches from two to four courses



71

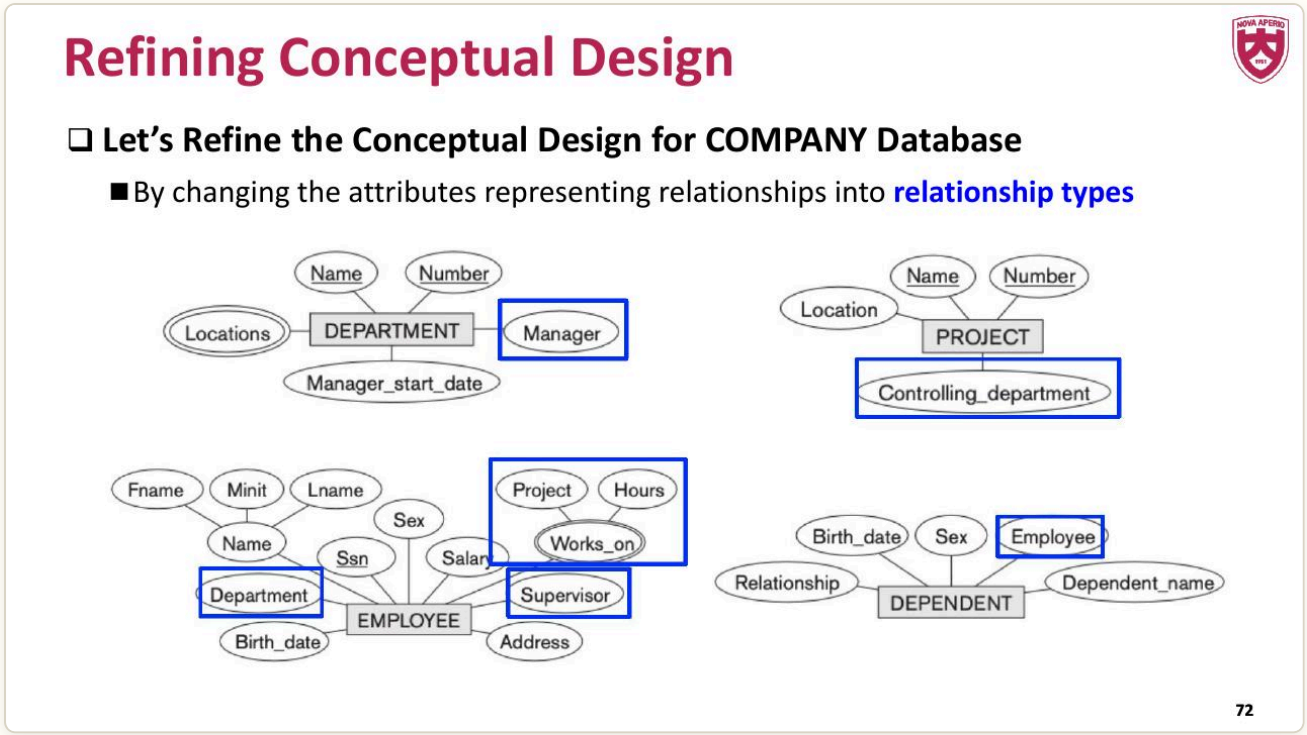
연습 2: ER 다이어그램의 제약

- 아래 그림에 (min, max) 제약을 적어라
  - 과목은 교재를 쓸 수도, 안 쓸 수도 있다
  - 과목은 교재를 **5권 넘게** 쓰지 않는다
  - 과목은 강사 **최소 1명, 최대 4명**이 가르친다
  - 교재는 **몇몇** 과목에서 쓰인다
  - 강사는 **2~4개** 과목을 가르친다
- 그림: INSTRUCTOR – TEACHES – COURSE – USES – TEXT

필기 ✍️

✏ 예.풀이 풀이 예시(교안에 정답 없음):
INSTRUCTOR (2,4) TEACHES (1,4) COURSE /
COURSE (0,5) USES (1,N) TEXT

💡 이해 문장 주어 엔티티 옆에 숫자를 적는다: '과목은 강사 1~4명' ⟶ COURSE 쪽 (1,4)



개념 설계 다듬기

- COMPANY DB의 개념 설계를 다듬자
  - **관계를 나타내던 속성을 관계 타입으로** 바꾸어서
- 그림(파란 상자): DEPARTMENT.Manager, PROJECT.Controlling_department, EMPLOYEE.Department·Supervisor·Works_on, DEPENDENT.Employee

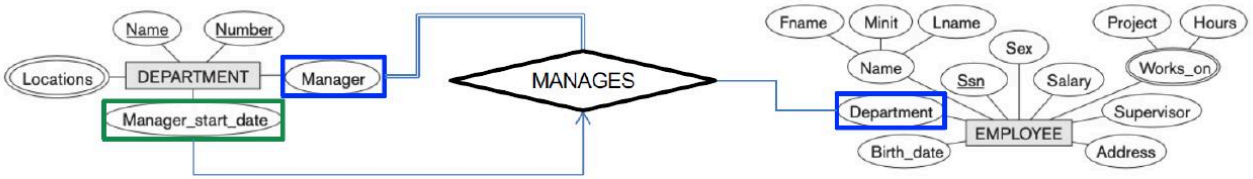
필기 ✍️

★ 시험 바뀔 속성 6개 ⟶ 관계 6개 (다음 쪽부터 하나씩)

Refining Conceptual Design: MANAGES

❑ **Relationship Type 1: MANAGES**

- Participating entity types: **EMPLOYEE** and **DEPARTMENT**
- Cardinality ratio: **1:1**
- EMPLOYEE participation is **partial**, DEPARTMENT participation is **total**
- The attribute **Manager_start_date** is assigned to the relationship type
  - ❑ *This attribute can be assigned to one of the DEPARTMENT and EMPLOYEE entity type



73

다듬기 1: MANAGES

- 참여: **EMPLOYEE**, **DEPARTMENT**
- 카디널리티: **1:1**
- EMPLOYEE 참여는 **partial**, DEPARTMENT 참여는 **total**
- **Manager_start_date** 속성은 관계 타입에 둔다
  - *DEPARTMENT나 EMPLOYEE 중 한쪽에 둘 수도 있음 (1:1이므로)

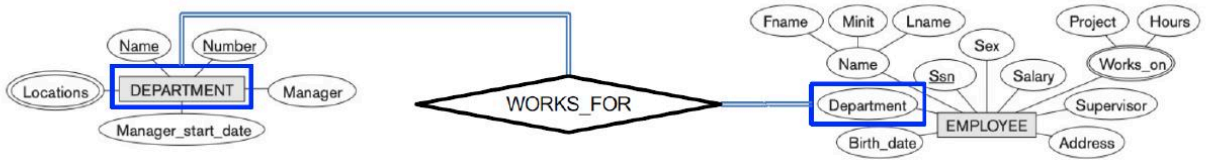
필기 ✍️

✏ 예.풀이 모든 부서엔 관리자가 있음(total), 모든 직원이 관리자는 아님(partial)

Refining Conceptual Design: WORKS_FOR

❑ **Relationship Type 2: WORKS_FOR**

- Participating entity types: **EMPLOYEE** and **DEPARTMENT**
- Cardinality ratio: **1:N**
  - ❑ *I.e.*, 1-side: DEPARTMENT, N-side: EMPLOYEE
- Both participations are total
- No relationship type attribute



74

다듬기 2: WORKS_FOR

- 참여: EMPLOYEE, DEPARTMENT
- 카디널리티: **1:N** — 1쪽 DEPARTMENT, N쪽 EMPLOYEE
- **둘 다 total** 참여
- 관계 속성 없음

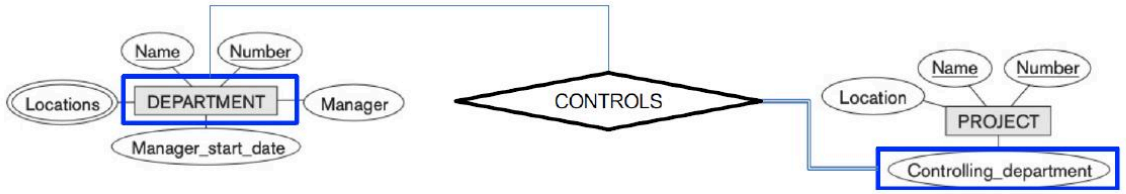
필기 ✍️

— 넘어가도 되는 쪽

Refining Conceptual Design: CONTROLS

❑ **Relationship Type 3: CONTROLS**

- Participating entity types: **DEPARTMENT** and **PROJECT**
- Cardinality ratio: **1:N**
  - ❑ *I.e.*, 1-side: DEPARTMENT, N-side: PROJECT
- PROJECT participation is **total**, DEPARTMENT participation is **partial**
- No relationship type attribute



75

다듬기 3: CONTROLS

- 참여: DEPARTMENT, PROJECT
- 카디널리티: **1:N** — 1쪽 DEPARTMENT, N쪽 PROJECT
- PROJECT 참여는 **total**, DEPARTMENT 참여는 **partial**
- 관계 속성 없음

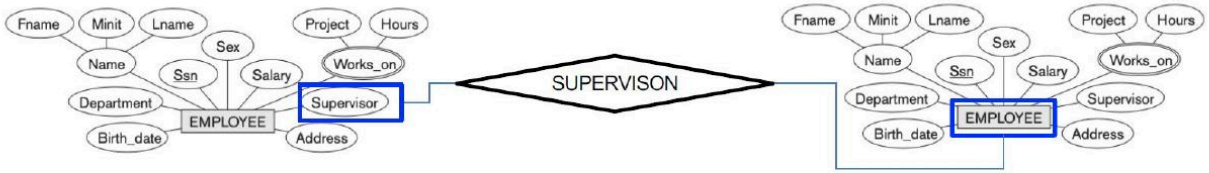
필기 ✍️

✏ 예.풀이 모든 프로젝트엔 관리 부서가 있지만, 프로젝트 없는 부서도 있을 수 있음

Refining Conceptual Design: SUPERVISION

❑ **Relationship Type 4: SUPERVISION**

- Participating entity types: EMPLOYEE
- Cardinality ratio: **1:N**
  - ❑ *I.e.*, 1-side: EMPLOYEE (in the supervisee role), N-side: EMPLOYEE (in the supervisor role)
- Both participations are **partial**
- No relationship type attribute



76

다듬기 4: SUPERVISION

- 참여: EMPLOYEE (한 타입이 두 번 — **순환 관계**)
- 카디널리티: **1:N**
  - 슬라이드: 1쪽 = supervisee 역할, N쪽 = supervisor 역할
- **둘 다 partial**
- 관계 속성 없음

필기 ✍️

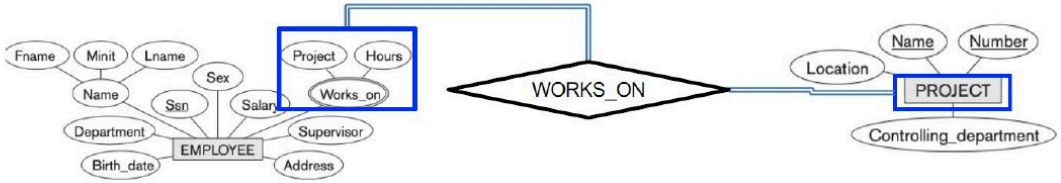
⚠ 주의 이 쪽 글은 역할이 뒤바뀐 것으로 보임: p.24·p.79 그림은 Supervisor 쪽 1, Supervisee 쪽 N, (min,max)도 Supervisor (0,N)·Supervisee (0,1). 상사 1명이 부하 N명 — 그림 기준으로 이해하고, 시험에선 교수님 표기 확인

★ 시험 둘 다 partial: 상사 없는 직원(사장), 부하 없는 직원 모두 있음

Refining Conceptual Design: WORKS_ON

❑ **Relationship Type 5: WORKS_ON**

- Participating entity types: **EMPLOYEE** and **PROJECT**
- Cardinality ratio: **M:N**
  - ❑ *I.e.*, an employee can work on several projects and a project can have several employees
- Both participations are **total**
  - ❑ An employee must work on at least one project; a project must be conducted by at least one employee
- The attribute **Hours** is assigned to the relationship type
  - ❑ It is determined by an employee-project combination



77

다듬기 5: WORKS_ON

- 참여: EMPLOYEE, PROJECT
- 카디널리티: **M:N** — 직원은 여러 프로젝트, 프로젝트는 여러 직원
- **둘 다 total** — 직원은 최소 한 프로젝트, 프로젝트는 최소 한 직원
- **Hours** 속성은 관계 타입에 — 직원–프로젝트 조합으로 정해지므로

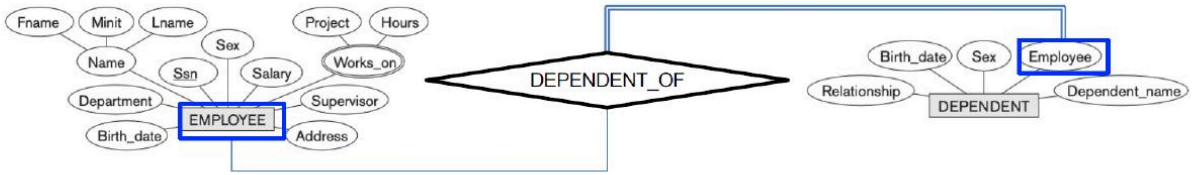
필기 ✍️

★ 시험 M:N + 관계 속성 Hours = 시험 단골 조합

Refining Conceptual Design: DEPENDENT_OF

❑ **Relationship Type 6: DEPENDENTS_OF**

- Participating entity types: **EMPLOYEE** and **DEPENDENT**
  - **Identifying entity type**: EMPLOYEE, **weak entity type**: DEPENDENT
  - The DEPENDENTS_OF is the identifying relationship for the weak entity type DEPENDENT
- Cardinality ratio: **1:N**
  - 1-side: EMPLOYEE, N-side: DEPENDENT (i.e., an employee can have several dependents)
- EMPLOYEE participation is **partial**, DEPENDENT participation is **total**
  - *I.e.*, an employee may or may not have dependents, but a dependent must have its employee



78

다듬기 6: DEPENDENTS_OF

- 참여: EMPLOYEE, DEPENDENT
  - **식별 엔티티 타입: EMPLOYEE**, **약한 엔티티 타입: DEPENDENT**
  - DEPENDENTS_OF가 DEPENDENT의 **식별 관계**
- 카디널리티: **1:N** — 1쪽 EMPLOYEE, N쪽 DEPENDENT
- EMPLOYEE 참여 **partial**, DEPENDENT 참여 **total**
  - 직원은 부양가족이 없을 수도 있지만, 부양가족은 반드시 그 직원이 있다

필기 ✍️

★ 시험 약한 엔티티 쪽은 항상 total — 여기서도 DEPENDENT total

Refining Conceptual Design: Result

❑ **Process of Refinement**

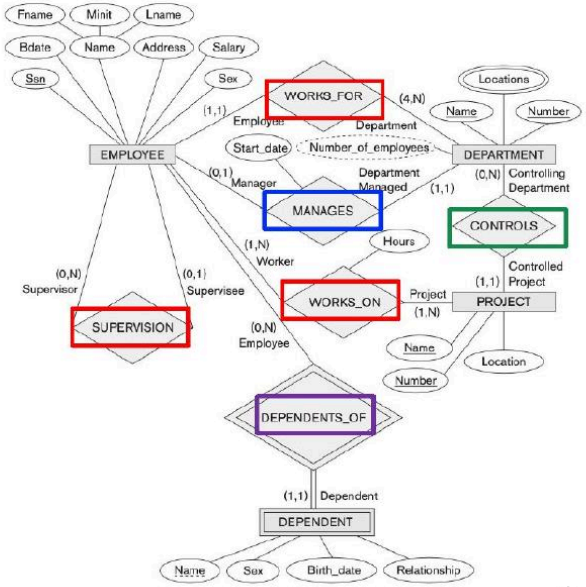
- Specifying all relationship types
- Changing the attributes representing relationships into **relationship types**

Manager and Manager_start_date from DEPARTMENT → To MANAGES

Controlling_department from PROJECT → To CONTROLS

Department, Supervisor, and Works_on from EMPLOYEE → To WORKS_FOR, EMPLOYEE, and WORKS_ON

Employee from DEPENDENT → To DEPENDENTS_OF



79

개념 설계 다듬기: 결과

- 다듬는 과정
  - 모든 관계 타입을 정하고
  - 관계를 나타내던 속성을 **관계 타입으로** 바꾼다
- 옮긴 것
  - DEPARTMENT의 Manager, Manager_start_date → **MANAGES**
  - PROJECT의 Controlling_department → **CONTROLS**
  - EMPLOYEE의 Department, Supervisor, Works_on → **WORKS_FOR, SUPERVISION, WORKS_ON**
  - DEPENDENT의 Employee → **DEPENDENTS_OF**
- 그림(min,max): WORKS_FOR EMPLOYEE (1,1)·DEPARTMENT (4,N) / MANAGES (0,1)·(1,1) / CONTROLS (0,N)·(1,1) / WORKS_ON (1,N)·(1,N) / SUPERVISION Supervisor (0,N)·Supervisee (0,1) / DEPENDENTS_OF (0,N)·(1,1)

필기 ✍️

★ 시험 완성 ER 다이어그램을 직접 그려 볼 수 있으면 ER 파트 끝

✏ 예.풀이 DEPARTMENT (4,N) in WORKS_FOR = 부서마다 직원 최소 4명이라는 그림 속 제약

💡 이해 슬라이드 상자의 'EMPLOYEE'는 SUPERVISION을 뜻함 (Supervisor 속성 → SUPERVISION)

Summary: Entity-Relationship (ER) Model

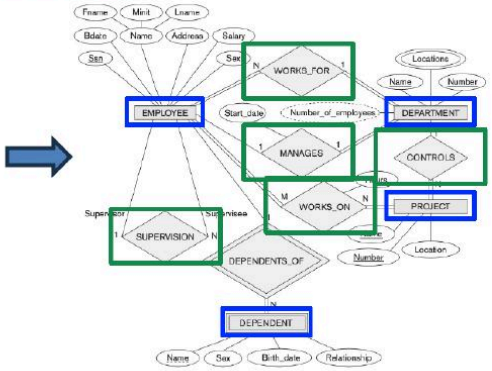
❑ **Describe Data as a Collection of *Entities* and *Relationships***

- **Entity**: a "thing" or "object" that is distinguishable from others
- **Relationship**: an association among several entities

❑ **Focus on the Schema, Rather Than the Instances**

❑ **What Information Should be Managed?**

- We want to keep track of a company's employees, departments, and projects
- **Department**
  - The company is organized into '**department**'s
    - Each has a unique name, a unique number, locations, manager, start date when that manager began, ...
- **Project**
  - A department can control a number of '**project**'s
    - Each has a unique name, a unique number, a single location, controlling department
- **Employee**
  - '**Employee**'s are working in the company
    - Each has a name, SSN, address, salary, gender, birth date, projects that the employee works on, ...
- **Dependent**
  - For insurance purposes, the '**dependent**'s of each employee are needed
    - Each has a first name, gender, birth date, relationships to the employee, ...



Requirement Analysis

80

요약: ER 모델

- 데이터를 **엔티티와 관계의 모음**으로 기술
  - Entity: 다른 것과 구별되는 '것'·'객체'
  - Relationship: 여러 엔티티 사이의 연관
- **인스턴스보다 스키마에** 초점

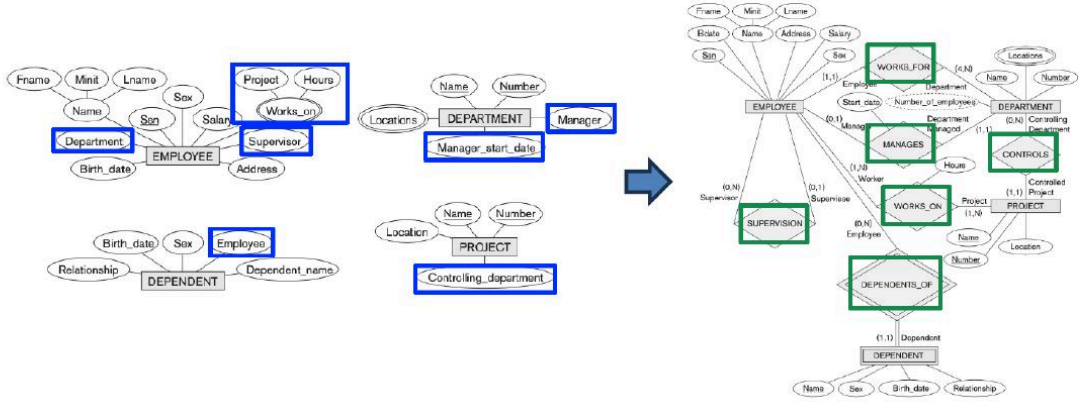
필기 ✍️

★ 시험 Focus on the Schema, rather than the instances

Summary: Designing ER Schema

❑ **Process of Designing ER Schema Diagram**

- ■(1) Describe entity types and their attributes based on the data requirements
- ■(2) **Specify relationship types and their constraints**
- ■(3) **Change the attributes representing relationships into relationship types**



81

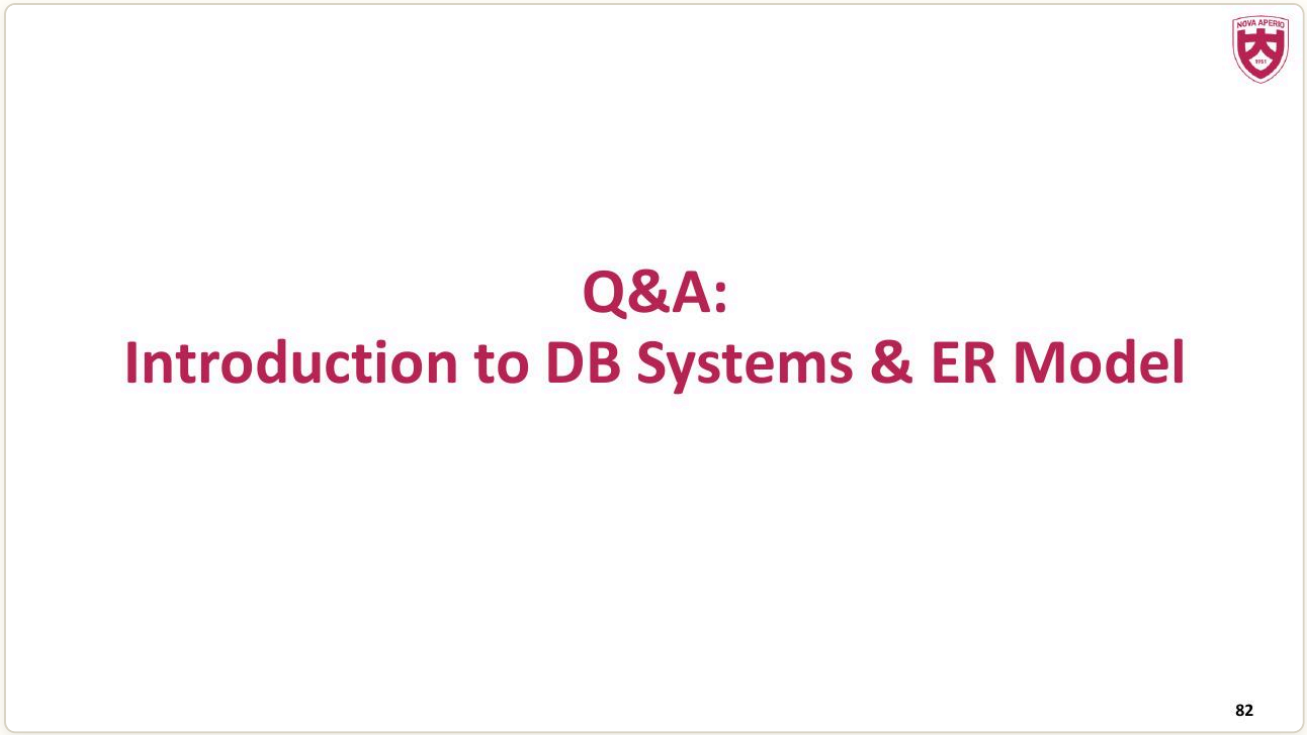
요약: ER 스키마 설계

- ER 스키마 다이어그램 설계 과정
  - (1) 요구사항에 따라 **엔티티 타입과 속성** 기술
  - (2) **관계 타입과 제약** 지정
  - (3) 관계를 나타내던 **속성을 관계 타입으로** 바꾸기

필기 ✍️

★ 시험 설계 3단계 순서 서술형 대비

💡 이해 문제 풀 때: 명사 → 엔티티, 'unique' → 키, 동사 → 관계, '하나/여럿' → 카디널리티, '반드시' → total



Q&A

- Introduction to DB Systems & ER Model — 끝

필기 ✍️

💡 이해 다음: Practice 1 (ER 모델 연습) · Lecture 3 관계형 모델